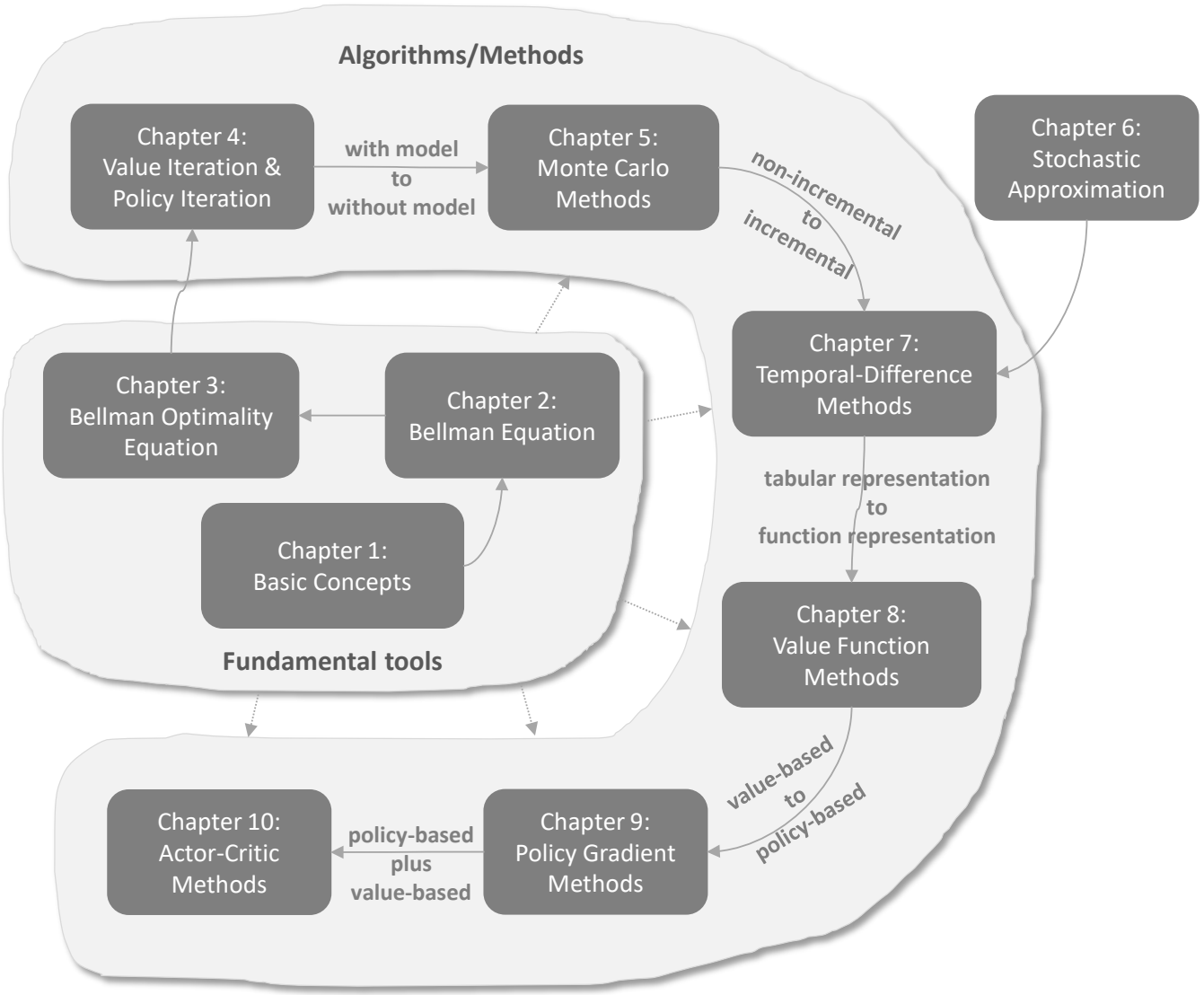
第八讲：值函数方法

April 11, 2026

大纲


Algorithms/Methods
Chapter 4: Value Iteration & Policy Iteration
with model to without model
Chapter 5: Monte Carlo Methods
non-incremental to incremental
Chapter 6: Stochastic Approximation
Chapter 7: Temporal-Difference Methods
tabular representation to function representation
Chapter 8: Value Function Methods
value-based to policy-based
Chapter 9: Policy Gradient Methods
policy-based plus value-based
Chapter 10: Actor-Critic Methods
Chapter 3: Bellman Optimality Equation
Chapter 2: Bellman Equation
Chapter 1: Basic Concepts
Fundamental tools

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

动机示例：从表格到函数

在本书中，状态值和动作值迄今均以表格形式表示。

- 例如，状态值：

状态	s_1	s_2	$\cdots$	s_n
值	$v_\pi(s_1)$	$v_\pi(s_2)$	$\cdots$	$v_\pi(s_n)$

- 例如，动作值：

	a_1	a_2	a_3	a_4	a_5
s_1	$q_\pi(s_1, a_1)$	$q_\pi(s_1, a_2)$	$q_\pi(s_1, a_3)$	$q_\pi(s_1, a_4)$	$q_\pi(s_1, a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$q_\pi(s_9, a_1)$	$q_\pi(s_9, a_2)$	$q_\pi(s_9, a_3)$	$q_\pi(s_9, a_4)$	$q_\pi(s_9, a_5)$

- 优点：直观，易于分析
- 缺点：难以处理大型或连续的状态或动作空间。体现在两个方面：1) 存储；2) 泛化能力

动机示例：从表格到函数

在本书中，状态值和动作值迄今均以表格形式表示。

- 例如，状态值：

状态	s_1	s_2	$\cdots$	s_n
值	$v_\pi(s_1)$	$v_\pi(s_2)$	$\cdots$	$v_\pi(s_n)$

- 例如，动作值：

	a_1	a_2	a_3	a_4	a_5
s_1	$q_\pi(s_1, a_1)$	$q_\pi(s_1, a_2)$	$q_\pi(s_1, a_3)$	$q_\pi(s_1, a_4)$	$q_\pi(s_1, a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$q_\pi(s_9, a_1)$	$q_\pi(s_9, a_2)$	$q_\pi(s_9, a_3)$	$q_\pi(s_9, a_4)$	$q_\pi(s_9, a_5)$

- 优点：直观，易于分析
- 缺点：难以处理大型或连续的状态或动作空间。体现在两个方面：1) 存储；2) 泛化能力

动机示例：从表格到函数

在本书中，状态值和动作值迄今均以表格形式表示。

- 例如，状态值：

状态	s_1	s_2	$\cdots$	s_n
值	$v_\pi(s_1)$	$v_\pi(s_2)$	$\cdots$	$v_\pi(s_n)$

- 例如，动作值：

	a_1	a_2	a_3	a_4	a_5
s_1	$q_\pi(s_1, a_1)$	$q_\pi(s_1, a_2)$	$q_\pi(s_1, a_3)$	$q_\pi(s_1, a_4)$	$q_\pi(s_1, a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$q_\pi(s_9, a_1)$	$q_\pi(s_9, a_2)$	$q_\pi(s_9, a_3)$	$q_\pi(s_9, a_4)$	$q_\pi(s_9, a_5)$

- 优点：直观，易于分析
- 缺点：难以处理大型或连续的状态或动作空间。体现在两个方面：1) 存储；2) 泛化能力

动机示例：从表格到函数

在本书中，状态值和动作值迄今均以表格形式表示。

- 例如，状态值：

状态	s_1	s_2	$\cdots$	s_n
值	$v_\pi(s_1)$	$v_\pi(s_2)$	$\cdots$	$v_\pi(s_n)$

- 例如，动作值：

	a_1	a_2	a_3	a_4	a_5
s_1	$q_\pi(s_1, a_1)$	$q_\pi(s_1, a_2)$	$q_\pi(s_1, a_3)$	$q_\pi(s_1, a_4)$	$q_\pi(s_1, a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$q_\pi(s_9, a_1)$	$q_\pi(s_9, a_2)$	$q_\pi(s_9, a_3)$	$q_\pi(s_9, a_4)$	$q_\pi(s_9, a_5)$

- 优点：直观，易于分析
- 缺点：难以处理大型或连续的状态或动作空间。体现在两个方面：1) 存储；2) 泛化能力

动机示例：从表格到函数

在本书中，状态值和动作值迄今均以表格形式表示。

- 例如，状态值：

状态	s_1	s_2	$\cdots$	s_n
值	$v_\pi(s_1)$	$v_\pi(s_2)$	$\cdots$	$v_\pi(s_n)$

- 例如，动作值：

	a_1	a_2	a_3	a_4	a_5
s_1	$q_\pi(s_1, a_1)$	$q_\pi(s_1, a_2)$	$q_\pi(s_1, a_3)$	$q_\pi(s_1, a_4)$	$q_\pi(s_1, a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$q_\pi(s_9, a_1)$	$q_\pi(s_9, a_2)$	$q_\pi(s_9, a_3)$	$q_\pi(s_9, a_4)$	$q_\pi(s_9, a_5)$

- 优点：直观，易于分析
- 缺点：难以处理大型或连续的状态或动作空间。体现在两个方面：1) 存储；2) 泛化能力

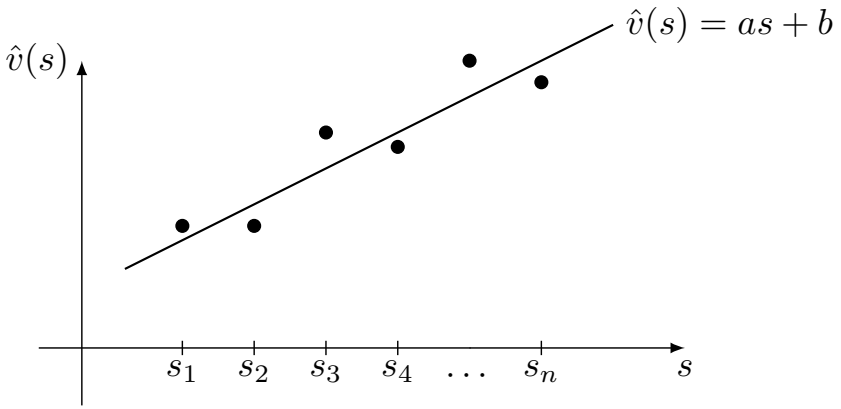
动机示例：从表格到函数

考虑一个例子：

- 共有n 个状态：$s_1, \ldots, s_n$。
- 状态值为$v_\pi(s_1), \ldots, v_\pi(s_n)$，其中$\pi$ 是给定策略。
- n 非常大！
- 我们希望用一条简单曲线来近似这些值。

动机示例：从表格到函数

例如，可以用一条简单的直线来拟合这些点。



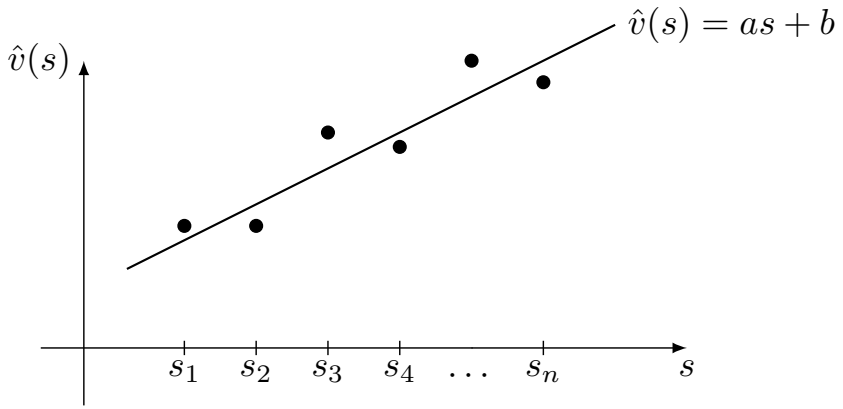
假设直线方程为：

$$\hat{v}(s, w) = as + b = \underbrace{[s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{w} = \phi^T(s) w$$

w 是参数向量；$\phi(s)$ 是s 的特征向量；$\hat{v}(s, w)$ 关于w 是线性的。

动机示例：从表格到函数

例如，可以用一条简单的直线来拟合这些点。



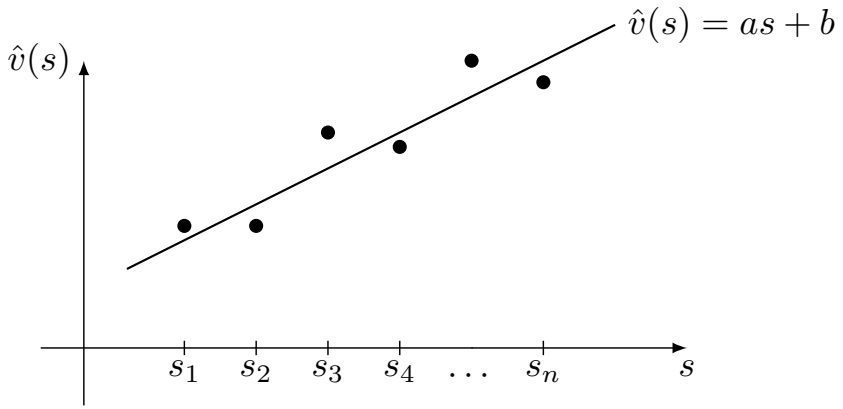
假设直线方程为：

$$\hat{v}(s, w) = as + b = \underbrace{[s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{w} = \phi^T(s) w$$

w 是参数向量；$\phi(s)$ 是s 的特征向量；$\hat{v}(s, w)$ 关于w 是线性的。

动机示例：从表格到函数

例如，可以用一条简单的直线来拟合这些点。



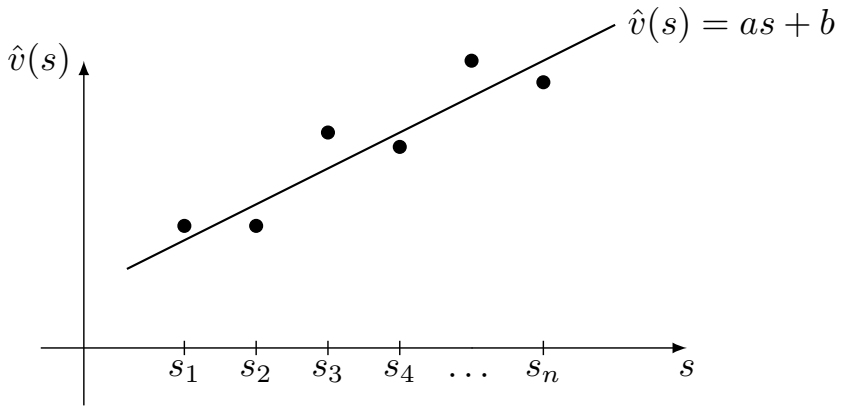
假设直线方程为：

$$\hat{v}(s, w) = as + b = \underbrace{[s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{w} = \phi^T(s) w$$

w 是参数向量；$\phi(s)$ 是s 的特征向量；$\hat{v}(s, w)$ 关于w 是线性的。

动机示例：从表格到函数

例如，可以用一条简单的直线来拟合这些点。



假设直线方程为：

$$\hat{v}(s, w) = as + b = \underbrace{[s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{w} = \phi^T(s) w$$

w 是参数向量；$\phi(s)$ 是s 的特征向量；$\hat{v}(s, w)$ 关于w 是线性的。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别一：如何读取状态的值

- 当值以表格形式表示时，可以直接从表中读取。
- 当值以函数形式表示时，需要将状态索引s 输入函数并计算函数值。

例如，$s \to \phi(s) \to \phi^T(s)w = \hat{v}(s, w)$

- 优点：节省存储。无需存储$|\mathcal{S}|$ 个状态值，只需存储维度较低的w。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别一：如何读取状态的值

- 当值以表格形式表示时，可以直接从表中读取。
- 当值以函数形式表示时，需要将状态索引s 输入函数并计算函数值。

例如，$s \to \phi(s) \to \phi^T(s)w = \hat{v}(s, w)$

- 优点：节省存储。无需存储$|\mathcal{S}|$ 个状态值，只需存储维度较低的w。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别一：如何读取状态的值

- 当值以表格形式表示时，可以直接从表中读取。
- 当值以函数形式表示时，需要将状态索引s 输入函数并计算函数值。

例如，$s \to \phi(s) \to \phi^T(s)w = \hat{v}(s, w)$

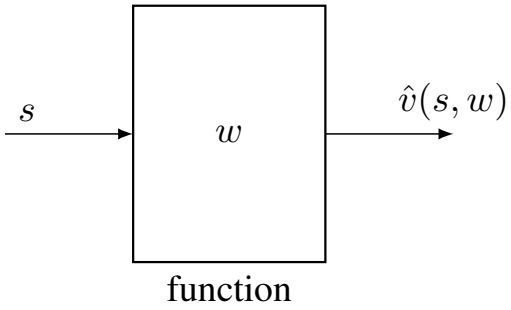
- 优点：节省存储。无需存储$|\mathcal{S}|$ 个状态值，只需存储维度较低的w。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别一：如何读取状态的值

- 当值以表格形式表示时，可以直接从表中读取。
- 当值以函数形式表示时，需要将状态索引s 输入函数并计算函数值。



例如，$s \to \phi(s) \to \phi^T(s)w = \hat{v}(s,w)$

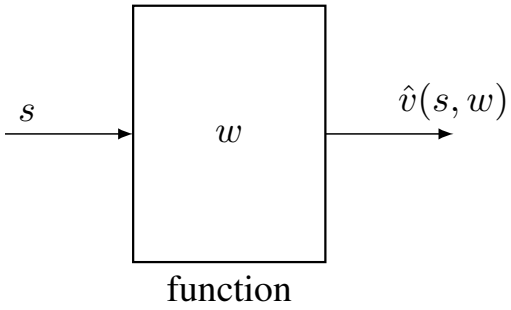
- 优点：节省存储。无需存储$|\mathcal{S}|$ 个状态值，只需存储维度较低的w。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别一：如何读取状态的值

- 当值以表格形式表示时，可以直接从表中读取。
- 当值以函数形式表示时，需要将状态索引s 输入函数并计算函数值。



例如，$s \to \phi(s) \to \phi^T(s)w = \hat{v}(s,w)$

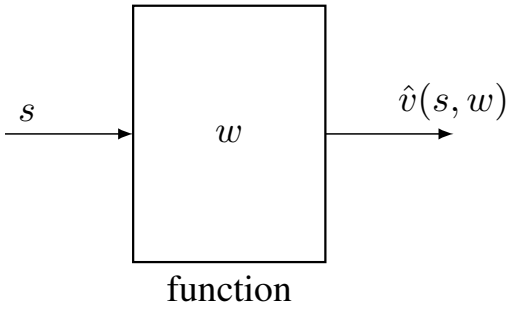
- 优点：节省存储。无需存储$|\mathcal{S}|$ 个状态值，只需存储维度较低的w。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别一：如何读取状态的值

- 当值以表格形式表示时，可以直接从表中读取。
- 当值以函数形式表示时，需要将状态索引s 输入函数并计算函数值。



例如，$s \to \phi(s) \to \phi^T(s)w = \hat{v}(s,w)$

- 优点：节省存储。无需存储$|\mathcal{S}|$ 个状态值，只需存储维度较低的w。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别二：如何更新状态的值

- 当值以表格形式表示时，可以直接修改表中的值。
- 当值以函数形式表示时，必须通过更新w 来间接改变值。
  - 如何更新w 将在后续详细介绍。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别二：如何更新状态的值

- 当值以表格形式表示时，可以直接修改表中的值。
- 当值以函数形式表示时，必须通过更新w 来间接改变值。
  - 如何更新w 将在后续详细介绍。

动机示例：从表格到函数

表格方法与函数方法的区别：

区别二：如何更新状态的值

- 当值以表格形式表示时，可以直接修改表中的值。
- 当值以函数形式表示时，必须通过更新w 来间接改变值。
  - 如何更新w 将在后续详细介绍。

动机示例：从表格到函数

表格方法与函数方法的区别：

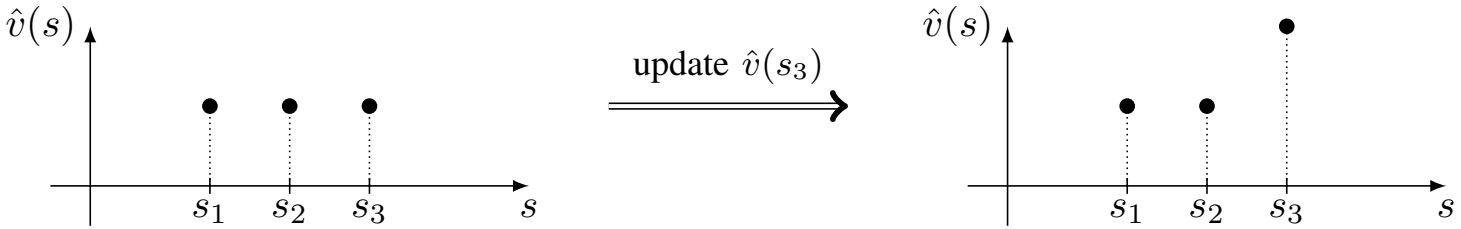
区别二：如何更新状态的值

- 当值以表格形式表示时，可以直接修改表中的值。
- 当值以函数形式表示时，必须通过更新w 来间接改变值。
  - 如何更新w 将在后续详细介绍。

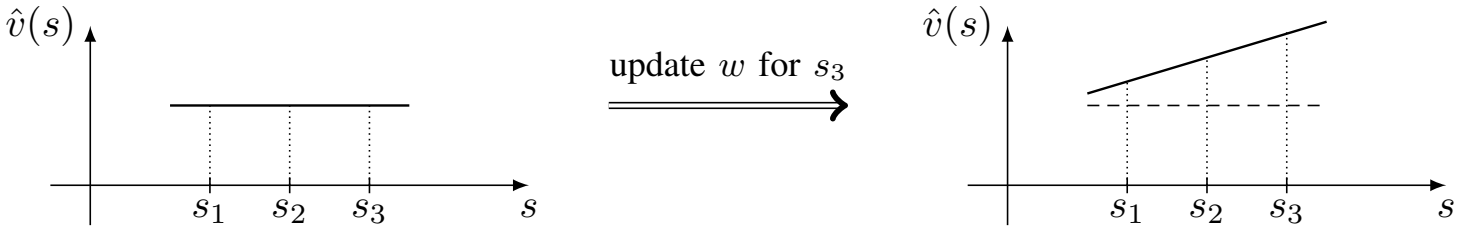
动机示例：从表格到函数

表格方法与函数方法的区别：

区别二：如何更新状态的值



(a) 表格方法



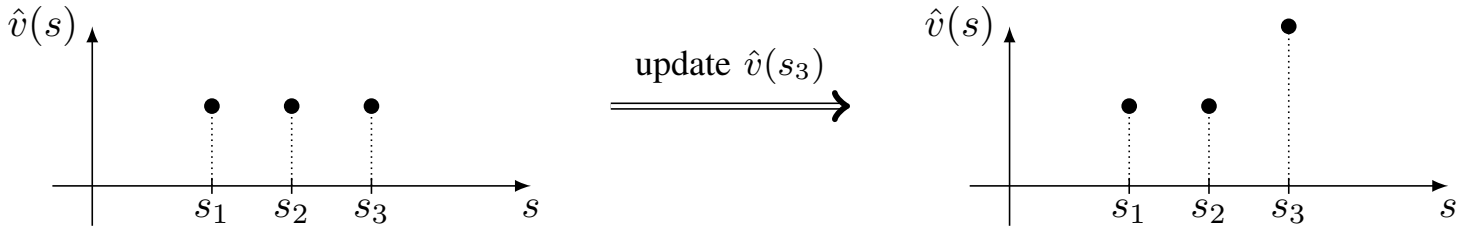
(b) 函数方法

优点：泛化能力。 当我们通过改变w 更新$\hat{v}(s, w)$ 时，邻近状态的值也会随之改变。

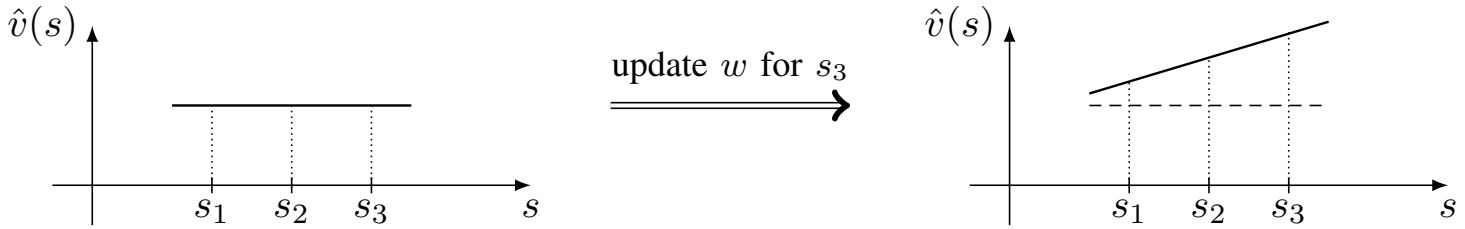
动机示例：从表格到函数

表格方法与函数方法的区别：

区别二：如何更新状态的值



(a) 表格方法



(b) 函数方法

优点：泛化能力。 当我们通过改变w 更新$\hat{v}(s, w)$ 时，邻近状态的值也会随之改变。

动机示例：从表格到函数

这些优点并非免费，代价是：状态值无法被精确表示，这也是该方法被称为近似的原因。

可以使用高阶曲线来更精确地拟合数据点：

$$\hat{v}(s,w) = as^2 + bs + c = \underbrace{[s^2, s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \\ c \end{bmatrix}}_{w} = \phi^T(s)w.$$

在此情形下：

- w 和$\phi(s)$ 的维度增加，值的拟合精度可能更高。
- 尽管$\hat{v}(s,w)$ 关于s 是非线性的，但关于w 是线性的。非线性被包含在$\phi(s)$ 中。

动机示例：从表格到函数

这些优点并非免费，代价是：状态值无法被精确表示，这也是该方法被称为近似的原因。

可以使用高阶曲线来更精确地拟合数据点：

$$\hat{v}(s,w) = as^2 + bs + c = \underbrace{[s^2, s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \\ c \end{bmatrix}}_{w} = \phi^T(s)w.$$

在此情形下：

- w 和$\phi(s)$ 的维度增加，值的拟合精度可能更高。
- 尽管$\hat{v}(s,w)$ 关于s 是非线性的，但关于w 是线性的。非线性被包含在$\phi(s)$ 中。

动机示例：从表格到函数

这些优点并非免费，代价是：状态值无法被精确表示，这也是该方法被称为近似的原因。

可以使用高阶曲线来更精确地拟合数据点：

$$\hat{v}(s,w) = as^2 + bs + c = \underbrace{[s^2, s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \\ c \end{bmatrix}}_{w} = \phi^T(s)w.$$

在此情形下：

- w 和$\phi(s)$ 的维度增加，值的拟合精度可能更高。
- 尽管$\hat{v}(s,w)$ 关于s 是非线性的，但关于w 是线性的。非线性被包含在$\phi(s)$ 中。

动机示例：从表格到函数

这些优点并非免费，代价是：状态值无法被精确表示，这也是该方法被称为近似的原因。

可以使用高阶曲线来更精确地拟合数据点：

$$\hat{v}(s,w) = as^2 + bs + c = \underbrace{[s^2, s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \\ c \end{bmatrix}}_{w} = \phi^T(s)w.$$

在此情形下：

- w 和$\phi(s)$ 的维度增加，值的拟合精度可能更高。
- 尽管$\hat{v}(s,w)$ 关于s 是非线性的，但关于w 是线性的。非线性被包含在$\phi(s)$中。

动机示例：从表格到函数

简要总结：

- **思路：** 使用参数化函数近似状态值和动作值：$\hat{v}(s, w) \approx v_\pi(s)$，其中$w \in \mathbb{R}^m$ 为参数向量。
- **关键区别：** 如何读取和改变$v(s)$ 的值。
- 优点：
  1) **存储：** w 的维度可远小于$|\mathcal{S}|$。
  2) **泛化：** 当访问状态s 时，参数w 被更新，从而某些未访问状态的值也可被更新。

动机示例：从表格到函数

简要总结：

- **思路：** 使用参数化函数近似状态值和动作值：$\hat{v}(s,w) \approx v_\pi(s)$，其中$w \in \mathbb{R}^m$ 为参数向量。
- **关键区别：** 如何读取和改变$v(s)$ 的值。
- 优点：
  1) **存储：** w 的维度可远小于$|\mathcal{S}|$。
  2) **泛化：** 当访问状态s 时，参数w 被更新，从而某些未访问状态的值也可被更新。

动机示例：从表格到函数

简要总结：

- **思路**： 使用参数化函数近似状态值和动作值：$\hat{v}(s, w) \approx v_\pi(s)$，其中$w \in \mathbb{R}^m$ 为参数向量。
- **关键区别**： 如何读取和改变$v(s)$ 的值。
- **优点**：
  1) **存储**： w 的维度可远小于$|\mathcal{S}|$。
  2) **泛化**： 当访问状态s 时，参数w 被更新，从而某些未访问状态的值也可被更新。

动机示例：从表格到函数

简要总结：

- **思路**： 使用参数化函数近似状态值和动作值：$\hat{v}(s, w) \approx v_\pi(s)$，其中$w \in \mathbb{R}^m$ 为参数向量。
- **关键区别**： 如何读取和改变$v(s)$ 的值。
- **优点**：
  1) **存储**： w 的维度可远小于$|\mathcal{S}|$。
  2) **泛化**： 当访问状态s 时，参数w 被更新，从而某些未访问状态的值也可被更新。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

目标函数

以更正式的方式介绍:

- 设$v_\pi(s)$ 和$\hat{v}(s,w)$ 分别为真实状态值和估计状态值。
- 我们的目标是找到最优的w，使得$\hat{v}(s,w)$ 对每个s 都能最好地近似$v_\pi(s)$。
- 这是一个策略评估问题，后续将扩展至策略改进。

寻找最优w 需要两个步骤:

- 第一步：定义目标函数。
- 第二步：推导优化目标函数的算法。

目标函数

以更正式的方式介绍:

- 设$v_\pi(s)$ 和$\hat{v}(s,w)$ 分别为真实状态值和估计状态值。
- 我们的目标是找到最优的w，使得$\hat{v}(s,w)$ 对每个s 都能最好地近似$v_\pi(s)$。
- 这是一个策略评估问题，后续将扩展至策略改进。

寻找最优w 需要两个步骤:

- 第一步：定义目标函数。
- 第二步：推导优化目标函数的算法。

目标函数

以更正式的方式介绍:

- 设$v_\pi(s)$ 和$\hat{v}(s,w)$ 分别为真实状态值和估计状态值。
- 我们的目标是找到最优的w，使得$\hat{v}(s,w)$ 对每个s 都能最好地近似$v_\pi(s)$。
- 这是一个策略评估问题，后续将扩展至策略改进。

寻找最优w 需要两个步骤:

- 第一步：定义目标函数。
- 第二步：推导优化目标函数的算法。

目标函数

以更正式的方式介绍:

- 设$v_\pi(s)$ 和$\hat{v}(s,w)$ 分别为真实状态值和估计状态值。
- 我们的目标是找到最优的w，使得$\hat{v}(s,w)$ 对每个s 都能最好地近似$v_\pi(s)$。
- 这是一个策略评估问题，后续将扩展至策略改进。

寻找最优w 需要两个步骤:

- 第一步：定义目标函数。
- 第二步：推导优化目标函数的算法。

目标函数

以更正式的方式介绍:

- 设$v_\pi(s)$ 和$\hat{v}(s,w)$ 分别为真实状态值和估计状态值。
- 我们的目标是找到最优的w，使得$\hat{v}(s,w)$ 对每个s 都能最好地近似$v_\pi(s)$。
- 这是一个策略评估问题，后续将扩展至策略改进。

寻找最优w 需要两个步骤:

- 第一步：定义目标函数。
- 第二步：推导优化目标函数的算法。

目标函数

以更正式的方式介绍:

- 设$v_\pi(s)$ 和$\hat{v}(s,w)$ 分别为真实状态值和估计状态值。
- 我们的目标是找到最优的w，使得$\hat{v}(s,w)$ 对每个s 都能最好地近似$v_\pi(s)$。
- 这是一个策略评估问题，后续将扩展至策略改进。

寻找最优w 需要两个步骤:

- 第一步：定义目标函数。
- 第二步：推导优化目标函数的算法。

目标函数

以更正式的方式介绍:

- 设$v_\pi(s)$ 和$\hat{v}(s,w)$ 分别为真实状态值和估计状态值。
- 我们的目标是找到最优的w，使得$\hat{v}(s,w)$ 对每个s 都能最好地近似$v_\pi(s)$。
- 这是一个策略评估问题，后续将扩展至策略改进。

寻找最优w 需要两个步骤:

- 第一步：定义目标函数。
- 第二步：推导优化目标函数的算法。

目标函数

目标函数为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2].$$

- 我们的目标是找到使$J(w)$ 最小的最优w。
- 期望关于随机变量$S \in \mathcal{S}$。
  S 的概率分布是什么?
  - 这是新问题，此前尚未讨论过状态的概率分布。
  - 定义S 的概率分布有多种方式。

目标函数

目标函数为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2].$$

- 我们的目标是找到使$J(w)$ 最小的最优w。
- 期望关于随机变量$S \in \mathcal{S}$。
  S 的概率分布是什么?
  - 这是新问题，此前尚未讨论过状态的概率分布。
  - 定义S 的概率分布有多种方式。

目标函数

目标函数为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2].$$

- 我们的目标是找到使$J(w)$ 最小的最优w。
- 期望关于随机变量$S \in \mathcal{S}$。
  S 的概率分布是什么?
  - 这是新问题，此前尚未讨论过状态的概率分布。
  - 定义S 的概率分布有多种方式。

目标函数

目标函数为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2].$$

- 我们的目标是找到使$J(w)$ 最小的最优w。
- 期望关于随机变量$S \in \mathcal{S}$。
  S 的概率分布是什么?
  - 这是新问题，此前尚未讨论过状态的概率分布。
  - 定义S 的概率分布有多种方式。

目标函数

目标函数为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2].$$

- 我们的目标是找到使$J(w)$ 最小的最优w。
- 期望关于随机变量$S \in \mathcal{S}$。
  S 的概率分布是什么?
  - 这是新问题，此前尚未讨论过状态的概率分布。
  - 定义S 的概率分布有多种方式。

目标函数

第一种方式：使用均匀分布。

- 即将所有状态视为同等重要，每个状态的概率设为$1/|\mathcal{S}|$。
- 此时目标函数变为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} (v_\pi(s) - \hat{v}(s, w))^2.$$

- 缺点：
  - 各状态未必同等重要。 例如，某些状态可能很少被策略访问。因此，该方式未考虑给定策略下马尔可夫过程的真实动态。

目标函数

第一种方式：使用均匀分布。

- 即将所有状态视为同等重要，每个状态的概率设为$1/|\mathcal{S}|$。
- 此时目标函数变为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} (v_\pi(s) - \hat{v}(s, w))^2.$$

- **缺点：**
  - 各状态未必同等重要。 例如，某些状态可能很少被策略访问。因此，该方式未考虑给定策略下马尔可夫过程的真实动态。

目标函数

第一种方式：使用均匀分布。

- 即将所有状态视为同等重要，每个状态的概率设为$1/|\mathcal{S}|$。
- 此时目标函数变为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} (v_\pi(s) - \hat{v}(s, w))^2.$$

- 缺点：
  - 各状态未必同等重要。 例如，某些状态可能很少被策略访问。因此，该方式未考虑给定策略下马尔可夫过程的真实动态。

目标函数

第一种方式：使用均匀分布。

- 即将所有状态视为同等重要，每个状态的概率设为$1/|\mathcal{S}|$。
- 此时目标函数变为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} (v_\pi(s) - \hat{v}(s, w))^2.$$

- **缺点：**
  - 各状态未必同等重要。 例如，某些状态可能很少被策略访问。因此，该方式未考虑给定策略下马尔可夫过程的真实动态。

目标函数

第二种方式：使用稳态分布。

- 稳态分布是本课程中将频繁使用的重要概念，描述了马尔可夫过程的长期行为。
- 设$\{d_\pi(s)\}_{s\in\mathcal{S}}$ 为策略π 下马尔可夫过程的稳态分布。由定义，$d_\pi(s) \geq 0$ 且$\sum_{s\in\mathcal{S}} d_\pi(s) = 1$。
- 目标函数可改写为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \sum_{s\in\mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s,w))^2.$$

  该函数是一个加权平方误差。
- 被更频繁访问的状态具有更高的$d_\pi(s)$，因此在目标函数中的权重也更高。

目标函数

第二种方式：使用稳态分布。

- 稳态分布是本课程中将频繁使用的重要概念，描述了马尔可夫过程的长期行为。
- 设$\{d_\pi(s)\}_{s\in\mathcal{S}}$ 为策略π 下马尔可夫过程的稳态分布。由定义，$d_\pi(s) \geq 0$ 且$\sum_{s\in\mathcal{S}} d_\pi(s) = 1$。
- 目标函数可改写为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \sum_{s\in\mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s,w))^2.$$

  该函数是一个加权平方误差。
- 被更频繁访问的状态具有更高的$d_\pi(s)$，因此在目标函数中的权重也更高。

目标函数

第二种方式：使用稳态分布。

- 稳态分布是本课程中将频繁使用的重要概念，描述了马尔可夫过程的长期行为。
- 设$\{d_\pi(s)\}_{s\in\mathcal{S}}$ 为策略π 下马尔可夫过程的稳态分布。由定义，$d_\pi(s) \geq 0$ 且$\sum_{s\in\mathcal{S}} d_\pi(s) = 1$。
- 目标函数可改写为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \sum_{s\in\mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s,w))^2.$$

  该函数是一个加权平方误差。
- 被更频繁访问的状态具有更高的$d_\pi(s)$，因此在目标函数中的权重也更高。

目标函数

第二种方式：使用稳态分布。

- 稳态分布是本课程中将频繁使用的重要概念，描述了马尔可夫过程的长期行为。
- 设$\{d_\pi(s)\}_{s\in\mathcal{S}}$ 为策略π 下马尔可夫过程的稳态分布。由定义，$d_\pi(s) \geq 0$ 且$\sum_{s\in\mathcal{S}} d_\pi(s) = 1$。
- 目标函数可改写为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \sum_{s\in\mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s,w))^2.$$

  该函数是一个加权平方误差。
- 被更频繁访问的状态具有更高的$d_\pi(s)$，因此在目标函数中的权重也更高。

目标函数

第二种方式：使用稳态分布。

- 稳态分布是本课程中将频繁使用的重要概念，描述了马尔可夫过程的长期行为。
- 设$\{d_\pi(s)\}_{s\in\mathcal{S}}$ 为策略π 下马尔可夫过程的稳态分布。由定义，$d_\pi(s) \geq 0$ 且$\sum_{s\in\mathcal{S}} d_\pi(s) = 1$。
- 目标函数可改写为：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \sum_{s\in\mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s,w))^2.$$

  该函数是一个加权平方误差。
- 被更频繁访问的状态具有更高的$d_\pi(s)$，因此在目标函数中的权重也更高。

目标函数——稳态分布

关于稳态分布的进一步解释：

- 分布：状态的分布
- 稳态：长期行为
- 总结：智能体按某策略运行足够长时间后，其处于各状态的概率可由该分布描述。

注：

- 稳态分布也称为平稳分布或极限分布。
- 它对理解值函数方法至关重要。
- 对下一讲的策略梯度方法也很重要。

目标函数——稳态分布

关于稳态分布的进一步解释：

- 分布：状态的分布
- 稳态：长期行为
- 总结：智能体按某策略运行足够长时间后，其处于各状态的概率可由该分布描述。

注：

- 稳态分布也称为平稳分布或极限分布。
- 它对理解值函数方法至关重要。
- 对下一讲的策略梯度方法也很重要。

目标函数——稳态分布

关于稳态分布的进一步解释：

- 分布：状态的分布
- 稳态：长期行为
- 总结：智能体按某策略运行足够长时间后，其处于各状态的概率可由该分布描述。

注：

- 稳态分布也称为平稳分布或极限分布。
- 它对理解值函数方法至关重要。
- 对下一讲的策略梯度方法也很重要。

目标函数——稳态分布

关于稳态分布的进一步解释：

- 分布：状态的分布
- 稳态：长期行为
- 总结：智能体按某策略运行足够长时间后，其处于各状态的概率可由该分布描述。

注：

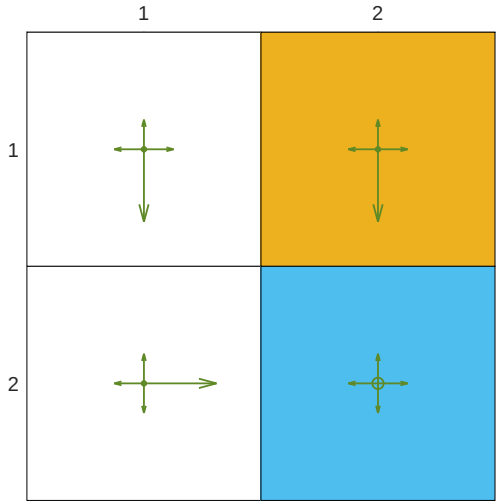
- 稳态分布也称为平稳分布或极限分布。
- 它对理解值函数方法至关重要。
- 对下一讲的策略梯度方法也很重要。

目标函数——稳态分布

示例说明：

- 给定如图所示的策略。
- 设$n_\pi(s)$ 为在由π 生成的极长回合中，s 被访问的次数。
- 则$d_\pi(s)$ 可近似为：

$$d_\pi(s) \approx \frac{n_\pi(s)}{\sum_{s' \in \mathcal{S}} n_\pi(s')}$$

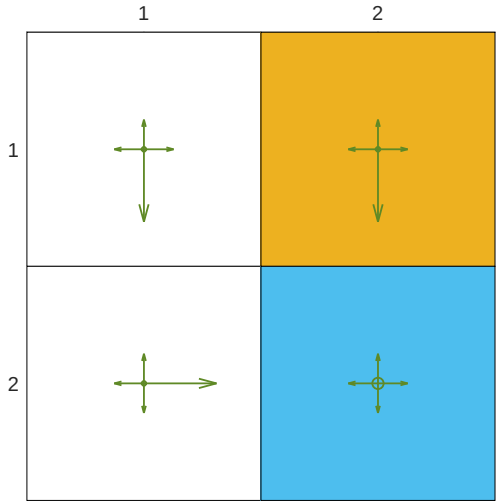


目标函数——稳态分布

示例说明:

- 给定如图所示的策略。
- 设$n_\pi(s)$ 为在由π 生成的极长回合中，s 被访问的次数。
- 则$d_\pi(s)$ 可近似为:

$$d_\pi(s) \approx \frac{n_\pi(s)}{\sum_{s' \in \mathcal{S}} n_\pi(s')}$$

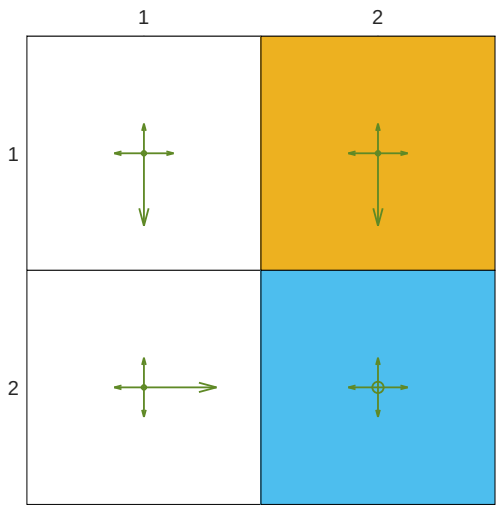


目标函数——稳态分布

示例说明：

- 给定如图所示的策略。
- 设$n_\pi(s)$ 为在由π 生成的极长回合中，s 被访问的次数。
- 则$d_\pi(s)$ 可近似为：

$$d_\pi(s) \approx \frac{n_\pi(s)}{\sum_{s' \in \mathcal{S}} n_\pi(s')}$$



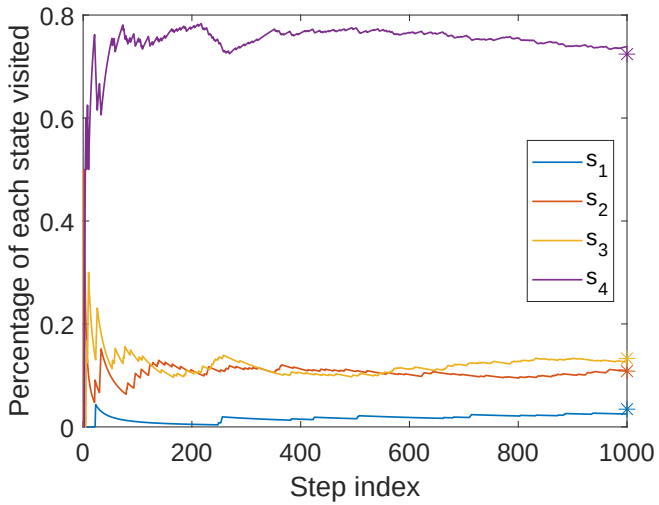


Figure: $\epsilon = 0.5$ 的ϵ-贪心策略的长期行为。

目标函数——稳态分布

收敛值可以预测，因为它们是d_π 的分量：

$$d_\pi^T = d_\pi^T P_\pi$$

在本例中，P_π 为：

$$P_\pi = \begin{bmatrix} 0.3 & 0.1 & 0.6 & 0 \\ 0.1 & 0.3 & 0 & 0.6 \\ 0.1 & 0 & 0.3 & 0.6 \\ 0 & 0.1 & 0.1 & 0.8 \end{bmatrix}.$$

可以计算得到特征值为1 的左特征向量为：

$$d_\pi = \left[0.0345, 0.1084, 0.1330, 0.7241\right]^T$$

详细介绍可参见教材。

目标函数——稳态分布

收敛值可以预测，因为它们是d_π 的分量：

$$d_\pi^T = d_\pi^T P_\pi$$

在本例中，P_π 为：

$$P_\pi = \begin{bmatrix} 0.3 & 0.1 & 0.6 & 0 \\ 0.1 & 0.3 & 0 & 0.6 \\ 0.1 & 0 & 0.3 & 0.6 \\ 0 & 0.1 & 0.1 & 0.8 \end{bmatrix}.$$

可以计算得到特征值为1 的左特征向量为：

$$d_\pi = \left[0.0345, 0.1084, 0.1330, 0.7241\right]^T$$

详细介绍可参见教材。

目标函数——稳态分布

收敛值可以预测，因为它们是d_π 的分量：

$$d_\pi^T = d_\pi^T P_\pi$$

在本例中，P_π 为：

$$P_\pi = \begin{bmatrix} 0.3 & 0.1 & 0.6 & 0 \\ 0.1 & 0.3 & 0 & 0.6 \\ 0.1 & 0 & 0.3 & 0.6 \\ 0 & 0.1 & 0.1 & 0.8 \end{bmatrix}.$$

可以计算得到特征值为1 的左特征向量为：

$$d_\pi = \Big[0.0345, 0.1084, 0.1330, 0.7241\Big]^T$$

详细介绍可参见教材。

目标函数——稳态分布

收敛值可以预测，因为它们是d_π 的分量：

$$d_\pi^T = d_\pi^T P_\pi$$

在本例中，P_π 为：

$$P_\pi = \begin{bmatrix} 0.3 & 0.1 & 0.6 & 0 \\ 0.1 & 0.3 & 0 & 0.6 \\ 0.1 & 0 & 0.3 & 0.6 \\ 0 & 0.1 & 0.1 & 0.8 \end{bmatrix}.$$

可以计算得到特征值为1 的左特征向量为：

$$d_\pi = \Big[0.0345, 0.1084, 0.1330, 0.7241\Big]^T$$

详细介绍可参见教材。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

优化算法

有了目标函数后，下一步是对其进行优化。

- 为最小化目标函数$J(w)$，可使用梯度下降算法:

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

真实梯度为:

$$\nabla_w J(w) = \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

优化算法

有了目标函数后，下一步是对其进行优化。

- 为最小化目标函数$J(w)$，可使用梯度下降算法:

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

真实梯度为:

$$\nabla_w J(w) = \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

优化算法

有了目标函数后，下一步是对其进行优化。

- 为最小化目标函数$J(w)$，可使用梯度下降算法：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

真实梯度为：

$$\nabla_w J(w) = \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

优化算法

有了目标函数后，下一步是对其进行优化。

- 为最小化目标函数$J(w)$，可使用梯度下降算法：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

真实梯度为：

$$\begin{aligned}\nabla_w J(w) &= \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] \\ &= \mathbb{E}[\nabla_w (v_\pi(S) - \hat{v}(S,w))^2]\end{aligned}$$

优化算法

有了目标函数后，下一步是对其进行优化。

- 为最小化目标函数$J(w)$，可使用梯度下降算法：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

真实梯度为：

$$\begin{aligned}\nabla_w J(w) &= \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] \\ &= \mathbb{E}[\nabla_w (v_\pi(S) - \hat{v}(S,w))^2] \\ &= 2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))(-\nabla_w \hat{v}(S,w))]\end{aligned}$$

优化算法

有了目标函数后，下一步是对其进行优化。

- 为最小化目标函数$J(w)$，可使用梯度下降算法：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

真实梯度为：

$$\begin{aligned}
\nabla_w J(w) &= \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] \\
&= \mathbb{E}[\nabla_w (v_\pi(S) - \hat{v}(S,w))^2] \\
&= 2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))(-\nabla_w \hat{v}(S,w))] \\
&= -2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))\nabla_w \hat{v}(S,w)]
\end{aligned}$$

优化算法

有了目标函数后，下一步是对其进行优化。

- 为最小化目标函数$J(w)$，可使用梯度下降算法：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

真实梯度为：

$$\begin{aligned}
\nabla_w J(w) &= \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] \\
&= \mathbb{E}[\nabla_w (v_\pi(S) - \hat{v}(S,w))^2] \\
&= 2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))(-\nabla_w \hat{v}(S,w))] \\
&= -2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))\nabla_w \hat{v}(S,w)]
\end{aligned}$$

上述真实梯度涉及期望的计算。

优化算法

可以用随机梯度替代真实梯度:

$$w_{k+1} = w_k + \alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))\nabla_w \hat{v}(S, w)]$$

$$\Downarrow$$

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

其中s_t 是S 的一个样本。此处将$2\alpha_t$ 合并为α_t。

- 样本应满足稳态分布，实践中可能并不满足。
- 该算法无法直接实现，因为它需要真实状态值v_π，而这正是待估计的未知量。
- 可以用近似值替换 $v_\pi(s_t)$，使算法可实现。

优化算法

可以用随机梯度替代真实梯度:

$$w_{k+1} = w_k + \alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))\nabla_w \hat{v}(S, w)]$$

$$\Downarrow$$

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

其中s_t 是S 的一个样本。此处将$2\alpha_t$ 合并为α_t。

- 样本应满足稳态分布，实践中可能并不满足。
- 该算法无法直接实现，因为它需要真实状态值v_π，而这正是待估计的未知量。
- 可以用近似值替换 $v_\pi(s_t)$，使算法可实现。

优化算法

可以用随机梯度替代真实梯度:

$$w_{k+1} = w_k + \alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))\nabla_w \hat{v}(S, w)]$$

$$\Downarrow$$

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

其中s_t 是S 的一个样本。此处将$2\alpha_t$ 合并为α_t。

- 样本应满足稳态分布，实践中可能并不满足。
- 该算法无法直接实现，因为它需要真实状态值v_π，而这正是待估计的未知量。
- 可以用近似值替换 $v_\pi(s_t)$，使算法可实现。

优化算法

可以用随机梯度替代真实梯度:

$$w_{k+1} = w_k + \alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))\nabla_w \hat{v}(S, w)]$$

$$\Downarrow$$

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

其中s_t 是S 的一个样本。此处将$2\alpha_t$ 合并为α_t。

- 样本应满足稳态分布，实践中可能并不满足。
- 该算法无法直接实现，因为它需要真实状态值v_π，而这正是待估计的未知量。
- 可以用近似值替换 $v_\pi(s_t)$，使算法可实现。

优化算法

具体地:

- 第一，**带函数近似的蒙特卡洛学习**
  设g_t 为从s_t 开始的折扣回报，则g_t 可用于近似$v_\pi(s_t)$，算法变为:

$$w_{t+1} = w_t + \alpha_t(g_t - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t).$$

- 第二，**带函数近似的TD 学习**
  根据TD 学习的思想，$r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t)$ 可视为$v_\pi(s_t)$ 的近似值，算法变为:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t).$$

优化算法

具体地:

- 第一，**带函数近似的蒙特卡洛学习**
  设g_t 为从s_t 开始的折扣回报，则g_t 可用于近似$v_\pi(s_t)$，算法变为:

$$w_{t+1} = w_t + \alpha_t(g_t - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t).$$

- 第二，**带函数近似的TD 学习**
  根据TD 学习的思想，$r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t)$ 可视为$v_\pi(s_t)$ 的近似值，算法变为:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t).$$

优化算法

具体地:

- 第一，**带函数近似的蒙特卡洛学习**
  设g_t 为从s_t 开始的折扣回报，则g_t 可用于近似$v_\pi(s_t)$，算法变为:

$$w_{t+1} = w_t + \alpha_t(g_t - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t).$$

- 第二，**带函数近似的TD 学习**
  根据TD 学习的思想，$r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t)$ 可视为$v_\pi(s_t)$ 的近似值，算法变为:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t).$$

优化算法

具体地:

- 第一，**带函数近似的蒙特卡洛学习**
  设g_t 为从s_t 开始的折扣回报，则g_t 可用于近似$v_\pi(s_t)$，算法变为:

$$w_{t+1} = w_t + \alpha_t(g_t - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t).$$

- 第二，**带函数近似的TD 学习**
  根据TD 学习的思想，$r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t)$ 可视为$v_\pi(s_t)$ 的近似值，算法变为:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t).$$

优化算法

具体地:

- 第一，**带函数近似的蒙特卡洛学习**
  设g_t 为从s_t 开始的折扣回报，则g_t 可用于近似$v_\pi(s_t)$，算法变为:

$$w_{t+1} = w_t + \alpha_t(g_t - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t).$$

- 第二，**带函数近似的TD 学习**
  根据TD 学习的思想，$r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t)$ 可视为$v_\pi(s_t)$ 的近似值，算法变为:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t).$$

优化算法

伪代码：带函数近似的状态值TD 学习

初始化： 关于w 可微的函数$\hat{v}(s,w)$，初始参数w_0。
目标： 学习给定策略π 的真实状态值。

对由π 生成的每个回合$\{(s_t, r_{t+1}, s_{t+1})\}_t$，执行：
    对每个样本(s_t, r_{t+1}, s_{t+1})，执行：
        一般情形：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t)$$

该算法只能估计给定策略的状态值，但它对理解后续介绍的其他算法非常重要。

优化算法

伪代码：带函数近似的状态值TD 学习

初始化： 关于w 可微的函数$\hat{v}(s,w)$，初始参数w_0。
目标： 学习给定策略π 的真实状态值。

对由π 生成的每个回合$\{(s_t, r_{t+1}, s_{t+1})\}_t$，执行：
对每个样本(s_t, r_{t+1}, s_{t+1})，执行：
一般情形：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t)$$

线性情形：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \phi^T(s_{t+1}) w_t - \phi^T(s_t) w_t\right] \phi(s_t)$$

该算法只能估计给定策略的状态值，但它对理解后续介绍的其他算法非常重要。

伪代码：带函数近似的状态值TD 学习

初始化： 关于w 可微的函数$\hat{v}(s,w)$，初始参数w_0。
目标： 学习给定策略π 的真实状态值。

对由π 生成的每个回合$\{(s_t, r_{t+1}, s_{t+1})\}_t$，执行：
对每个样本(s_t, r_{t+1}, s_{t+1})，执行：
一般情形：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t)$$

线性情形：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \phi^T(s_{t+1}) w_t - \phi^T(s_t) w_t\right] \phi(s_t)$$

该算法只能估计给定策略的状态值，但它对理解后续介绍的其他算法非常重要。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

函数近似器的选择

一个尚未回答的重要问题：如何选择函数$\hat{v}(s,w)$?

- 第一种方式，**以前广泛使用**：使用线性函数：

$$\hat{v}(s,w) = \phi^T(s)w$$

其中$\phi(s)$ 是特征向量，可以是多项式基、傅里叶基等（详见教材）。我们在动机示例和后续示例中均会用到。

- 第二种方式，**现今广泛使用**：使用神经网络作为非线性函数近似器。
  - 例如，输入为s，输出为$\hat{v}(s,w)$，参数为w。

函数近似器的选择

一个尚未回答的重要问题：如何选择函数$\hat{v}(s,w)$?

- 第一种方式，**以前广泛使用**：使用线性函数：

$$\hat{v}(s,w) = \phi^T(s)w$$

其中$\phi(s)$ 是特征向量，可以是多项式基、傅里叶基等（详见教材）。我们在动机示例和后续示例中均会用到。

- 第二种方式，**现今广泛使用**：使用神经网络作为非线性函数近似器。
  - 例如，输入为s，输出为$\hat{v}(s,w)$，参数为w。

函数近似器的选择

一个尚未回答的重要问题：如何选择函数$\hat{v}(s,w)$?

- 第一种方式，**以前广泛使用**：使用线性函数：

$$\hat{v}(s,w) = \phi^T(s)w$$

  其中$\phi(s)$ 是特征向量，可以是多项式基、傅里叶基等（详见教材）。我们在动机示例和后续示例中均会用到。

- 第二种方式，**现今广泛使用**：使用神经网络作为非线性函数近似器。
  - 例如，输入为s，输出为$\hat{v}(s,w)$，参数为w。

函数近似器的选择

一个尚未回答的重要问题：如何选择函数$\hat{v}(s,w)$?

- 第一种方式，**以前广泛使用**：使用线性函数：

$$\hat{v}(s,w) = \phi^T(s)w$$

  其中$\phi(s)$ 是特征向量，可以是多项式基、傅里叶基等（详见教材）。我们在动机示例和后续示例中均会用到。

- 第二种方式，**现今广泛使用**：使用神经网络作为非线性函数近似器。
  - 例如，输入为s，输出为$\hat{v}(s,w)$，参数为w。

函数近似器的选择

一个尚未回答的重要问题：如何选择函数$\hat{v}(s,w)$?

- 第一种方式，**以前广泛使用**：使用线性函数：

$$\hat{v}(s,w) = \phi^T(s)w$$

  其中$\phi(s)$ 是特征向量，可以是多项式基、傅里叶基等（详见教材）。我们在动机示例和后续示例中均会用到。

- 第二种方式，**现今广泛使用**：使用神经网络作为非线性函数近似器。
  - 例如，输入为s，输出为$\hat{v}(s,w)$，参数为w。

函数近似器的选择

一个尚未回答的重要问题：如何选择函数$\hat{v}(s,w)$?

- 第一种方式，**以前广泛使用**：使用线性函数：

$$\hat{v}(s,w) = \phi^T(s)w$$

  其中$\phi(s)$ 是特征向量，可以是多项式基、傅里叶基等（详见教材）。我们在动机示例和后续示例中均会用到。

- 第二种方式，**现今广泛使用**：使用神经网络作为非线性函数近似器。
  - 例如，输入为s，输出为$\hat{v}(s,w)$，参数为w。

线性函数近似

在$\hat{v}(s,w)=\phi^T(s)w$ 的线性情形下:

$$\nabla_w \hat{v}(s,w)=\phi(s).$$

将梯度代入TD 算法:

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)-\hat{v}(s_t,w_t)\right]\nabla_w\hat{v}(s_t,w_t)$$

得到:

$$w_{t+1}=w_t+\alpha_t\big[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\big]\phi(s_t),$$

即带线性函数近似的TD 学习算法，本课程中称为TD-Linear。

线性函数近似

在$\hat{v}(s,w)=\phi^T(s)w$ 的线性情形下:

$$\nabla_w \hat{v}(s,w)=\phi(s).$$

将梯度代入TD 算法:

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)-\hat{v}(s_t,w_t)\right]\nabla_w\hat{v}(s_t,w_t)$$

得到:

$$w_{t+1}=w_t+\alpha_t\big[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\big]\phi(s_t),$$

即带线性函数近似的TD 学习算法，本课程中称为TD-Linear。

线性函数近似

在$\hat{v}(s,w)=\phi^T(s)w$ 的线性情形下:

$$\nabla_w \hat{v}(s,w)=\phi(s).$$

将梯度代入TD 算法:

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)-\hat{v}(s_t,w_t)\right]\nabla_w\hat{v}(s_t,w_t)$$

得到:

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\right]\phi(s_t),$$

即带线性函数近似的TD 学习算法，本课程中称为TD-Linear。

线性函数近似

在$\hat{v}(s,w)=\phi^T(s)w$ 的线性情形下:

$$\nabla_w \hat{v}(s,w)=\phi(s).$$

将梯度代入TD 算法:

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)-\hat{v}(s_t,w_t)\right]\nabla_w\hat{v}(s_t,w_t)$$

得到:

$$w_{t+1}=w_t+\alpha_t\Big[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\Big]\phi(s_t),$$

即带线性函数近似的TD 学习算法，本课程中称为TD-Linear。

线性函数近似

在$\hat{v}(s,w)=\phi^T(s)w$ 的线性情形下:

$$\nabla_w \hat{v}(s,w)=\phi(s).$$

将梯度代入TD 算法:

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)-\hat{v}(s_t,w_t)\right]\nabla_w\hat{v}(s_t,w_t)$$

得到:

$$w_{t+1}=w_t+\alpha_t\Big[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\Big]\phi(s_t),$$

即带线性函数近似的TD 学习算法，本课程中称为TD-Linear。

线性函数近似

- 线性函数方法的**缺点**:
  - 难以选择合适的特征向量。

- 线性函数方法的**优点**:
  - 线性情形下TD 算法的理论性质远比非线性情形更易理解。
  - 线性函数近似仍然强大，因为表格表示是线性函数表示的特殊情形。

线性函数近似

- 线性函数方法的**缺点**:
  - 难以选择合适的特征向量。
- 线性函数方法的**优点**:
  - 线性情形下TD 算法的理论性质远比非线性情形更易理解。
  - 线性函数近似仍然强大，因为表格表示是线性函数表示的特殊情形。

线性函数近似

- 线性函数方法的**缺点**:
  - 难以选择合适的特征向量。

- 线性函数方法的**优点**:
  - 线性情形下TD 算法的理论性质远比非线性情形更易理解。
  - 线性函数近似仍然强大，因为表格表示是线性函数表示的特殊情形。

线性函数近似

接下来证明表格表示是线性函数表示的特殊情形，从而两种表示方式得到统一！

- 考虑状态s 的特殊特征向量：

$$\phi(s) = e_s \in \mathbb{R}^{|\mathcal{S}|},$$

其中e_s 是第s 个分量为1、其余分量为0 的向量。

- 在此情形下：

$$\hat{v}(s, w) = \phi^T(s)w = e_s^T w = w(s),$$

其中$w(s)$ 是w 的第s 个分量。

线性函数近似

接下来证明表格表示是线性函数表示的特殊情形，从而两种表示方式得到统一！

- 考虑状态s 的特殊特征向量：

$$\phi(s) = e_s \in \mathbb{R}^{|\mathcal{S}|},$$

其中e_s 是第s 个分量为1、其余分量为0 的向量。

- 在此情形下：

$$\hat{v}(s, w) = \phi^T(s)w = e_s^T w = w(s),$$

其中$w(s)$ 是w 的第s 个分量。

线性函数近似

接下来证明表格表示是线性函数表示的特殊情形，从而两种表示方式得到统一！

- 考虑状态s 的特殊特征向量：

$$\phi(s) = e_s \in \mathbb{R}^{|\mathcal{S}|},$$

其中e_s 是第s 个分量为1、其余分量为0 的向量。

- 在此情形下：

$$\hat{v}(s, w) = \phi^T(s)w = e_s^T w = w(s),$$

其中$w(s)$ 是w 的第s 个分量。

线性函数近似

回顾TD-Linear 算法:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \phi^T(s_{t+1}) w_t - \phi^T(s_t) w_t \right] \phi(s_t),$$

- 当$\phi(s_t) = e_s$ 时，上述算法变为:

$$w_{t+1} = w_t + \alpha_t \left(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t) \right) e_{s_t}.$$

  这是一个向量方程，仅更新w_t 的第s_t 个分量。
- 两侧同乘以$e_{s_t}^T$，得到:

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_t \left(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t) \right),$$

  这正是表格TD 算法（此处称为TD-Table）。

总结: 若选择特殊特征向量，TD-Linear 退化为TD-Table。

线性函数近似

回顾TD-Linear 算法:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \phi^T(s_{t+1}) w_t - \phi^T(s_t) w_t \right] \phi(s_t),$$

- 当$\phi(s_t) = e_s$ 时，上述算法变为:

$$w_{t+1} = w_t + \alpha_t \left(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t) \right) e_{s_t}.$$

  这是一个向量方程，仅更新w_t 的第s_t 个分量。

- 两侧同乘以$e_{s_t}^T$，得到:

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_t \left(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t) \right),$$

  这正是表格TD 算法（此处称为TD-Table）。

总结: 若选择特殊特征向量，TD-Linear 退化为TD-Table。

线性函数近似

回顾TD-Linear 算法:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \phi^T(s_{t+1}) w_t - \phi^T(s_t) w_t \right] \phi(s_t),$$

- 当$\phi(s_t) = e_s$ 时，上述算法变为:

$$w_{t+1} = w_t + \alpha_t \left(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t) \right) e_{s_t}.$$

  这是一个向量方程，仅更新w_t 的第s_t 个分量。

- 两侧同乘以$e_{s_t}^T$，得到:

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_t \left(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t) \right),$$

  这正是表格TD 算法（此处称为TD-Table）。

总结: 若选择特殊特征向量，TD-Linear 退化为TD-Table。

线性函数近似

回顾TD-Linear 算法:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \phi^T(s_{t+1}) w_t - \phi^T(s_t) w_t \right] \phi(s_t),$$

- 当$\phi(s_t) = e_s$ 时，上述算法变为:

$$w_{t+1} = w_t + \alpha_t \left(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t) \right) e_{s_t}.$$

  这是一个向量方程，仅更新w_t 的第s_t 个分量。

- 两侧同乘以$e_{s_t}^T$，得到:

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_t \left(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t) \right),$$

  这正是表格TD 算法（此处称为TD-Table）。

总结: 若选择特殊特征向量，TD-Linear 退化为TD-Table。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

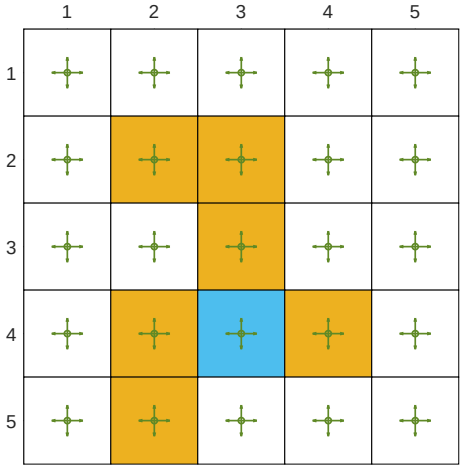
4 带函数近似的Q 学习

5 深度Q 学习

6 总结

示例

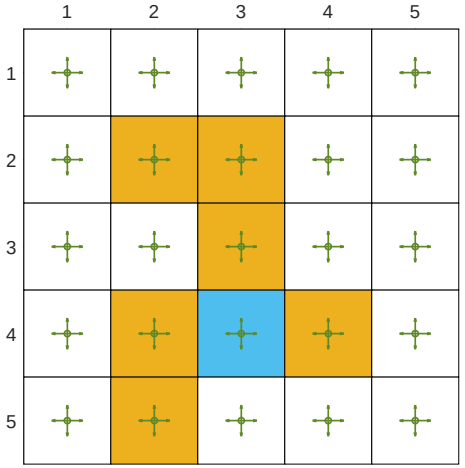
考虑一个5×5 网格世界示例：



- 给定策略：对任意s, a，$\pi(a|s) = 0.2$
- 目标：估计该策略的状态值（策略评估问题）。
- 共25 个状态值。接下来展示可以用少于25 个参数来近似25 个状态值。
- 设$r_{\text{forbidden}} = r_{\text{boundary}} = -1$，$r_{\text{target}} = 1$，$\gamma = 0.9$。

示例

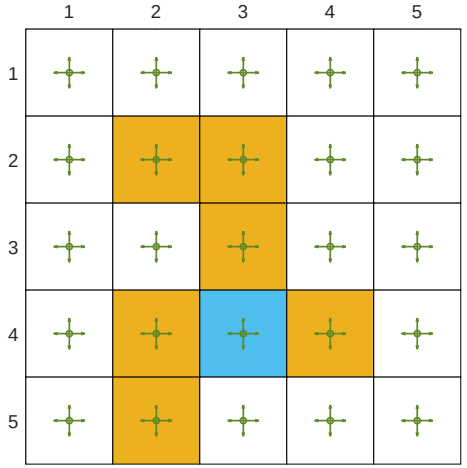
考虑一个5×5 网格世界示例：



- 给定策略：对任意s, a，$\pi(a|s) = 0.2$
- 目标：估计该策略的状态值（策略评估问题）。
- 共25 个状态值。接下来展示可以用少于25 个参数来近似25 个状态值。
- 设$r_{\text{forbidden}} = r_{\text{boundary}} = -1$，$r_{\text{target}} = 1$，$\gamma = 0.9$。

示例

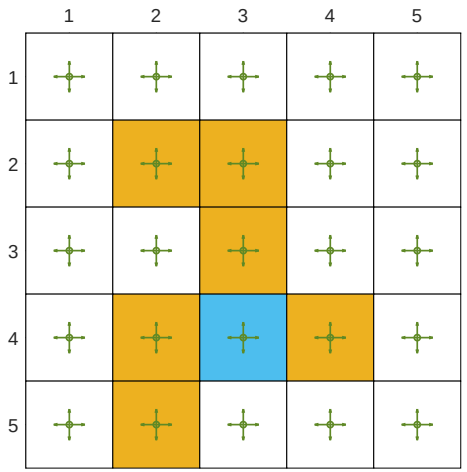
考虑一个5×5 网格世界示例：



- 给定策略：对任意s, a，$\pi(a|s) = 0.2$
- 目标：估计该策略的状态值（策略评估问题）。
- 共25 个状态值。接下来展示可以用少于25 个参数来近似25 个状态值。
- 设$r_{\text{forbidden}} = r_{\text{boundary}} = -1$，$r_{\text{target}} = 1$，$\gamma = 0.9$。

示例

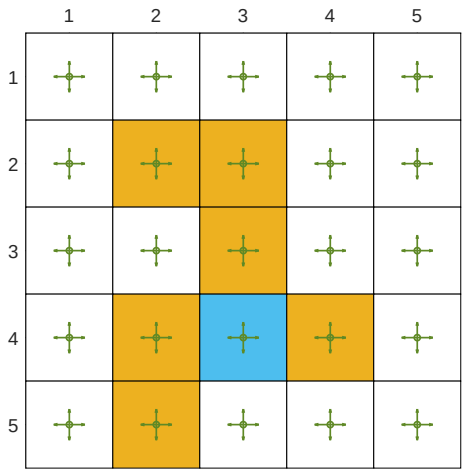
考虑一个5×5 网格世界示例：



- 给定策略：对任意s, a，$\pi(a|s) = 0.2$
- 目标：估计该策略的状态值（策略评估问题）。
- 共25 个状态值。接下来展示可以用少于25 个参数来近似25 个状态值。
- 设$r_{\text{forbidden}} = r_{\text{boundary}} = -1$，$r_{\text{target}} = 1$，$\gamma = 0.9$。

示例

考虑一个5×5 网格世界示例：

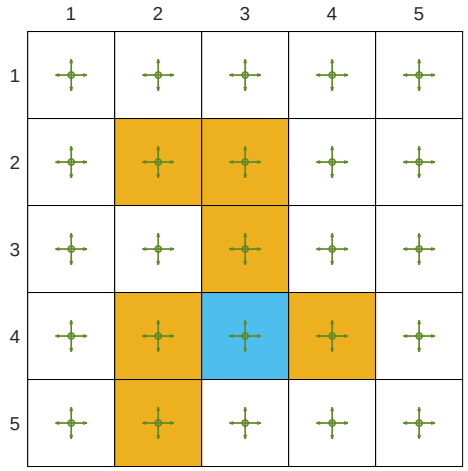


- 给定策略：对任意s, a，$\pi(a|s) = 0.2$
- 目标：估计该策略的状态值（策略评估问题）。
- 共25 个状态值。接下来展示可以用少于25 个参数来近似25 个状态值。
- 设$r_{\text{forbidden}} = r_{\text{boundary}} = -1$，$r_{\text{target}} = 1$，$\gamma = 0.9$。

示例

真实值:

- 真实状态值及其三维可视化



	1	2	3	4	5
1	-3.8	-3.8	-3.6	-3.1	-3.2
2	-3.8	-3.8	-3.8	-3.1	-2.9
3	-3.6	-3.9	-3.4	-3.2	-2.9
4	-3.9	-3.6	-3.4	-2.9	-3.2
5	-4.5	-4.2	-3.4	-3.4	-3.5

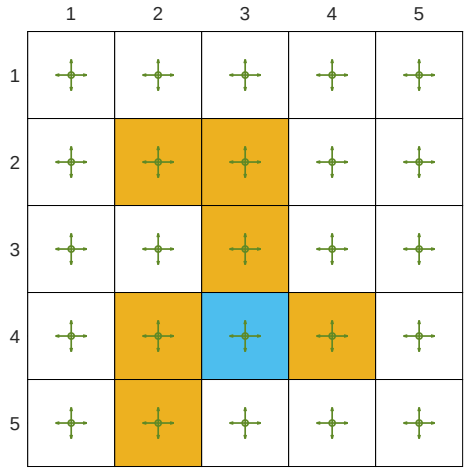
经验样本:

- 按给定策略生成了500 个回合。
- 每个回合有500 步，起始状态-动作对从均匀分布中随机选取。

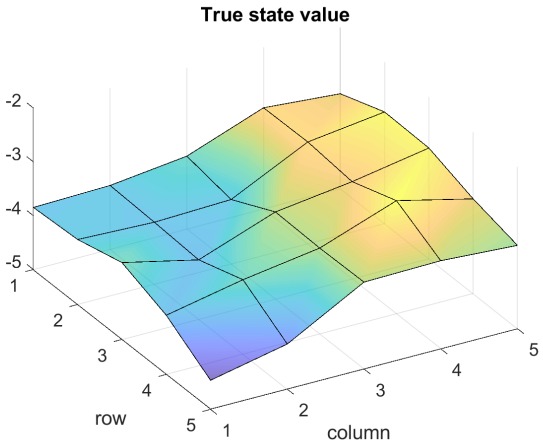
示例

真实值:

- 真实状态值及其三维可视化



	1	2	3	4	5
1	-3.8	-3.8	-3.6	-3.1	-3.2
2	-3.8	-3.8	-3.8	-3.1	-2.9
3	-3.6	-3.9	-3.4	-3.2	-2.9
4	-3.9	-3.6	-3.4	-2.9	-3.2
5	-4.5	-4.2	-3.4	-3.4	-3.5



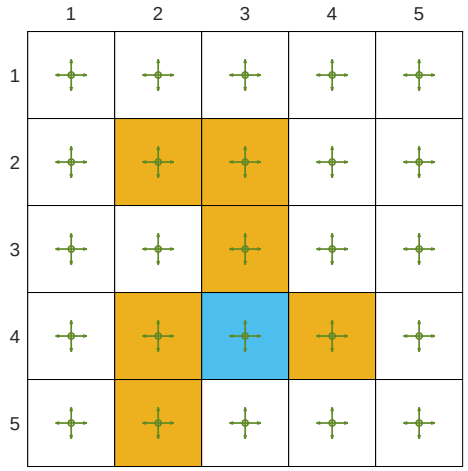
经验样本:

- 按给定策略生成了500 个回合。
- 每个回合有500 步，起始状态-动作对从均匀分布中随机选取。

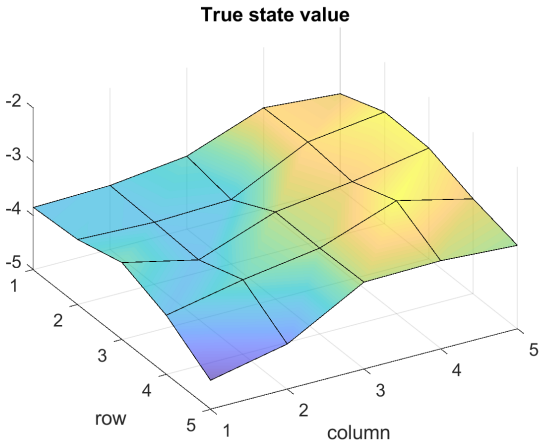
示例

真实值:

- 真实状态值及其三维可视化



	1	2	3	4	5
1	-3.8	-3.8	-3.6	-3.1	-3.2
2	-3.8	-3.8	-3.8	-3.1	-2.9
3	-3.6	-3.9	-3.4	-3.2	-2.9
4	-3.9	-3.6	-3.4	-2.9	-3.2
5	-4.5	-4.2	-3.4	-3.4	-3.5



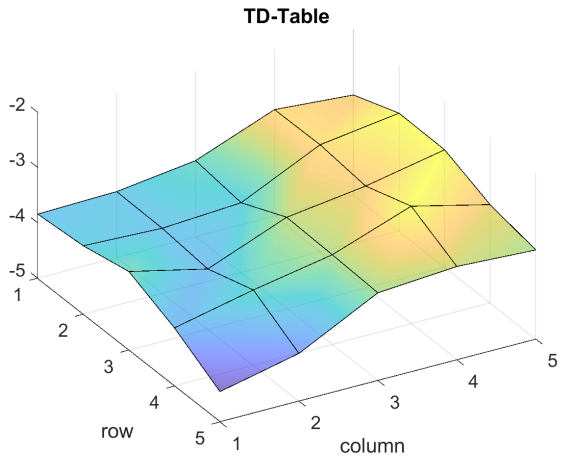
经验样本:

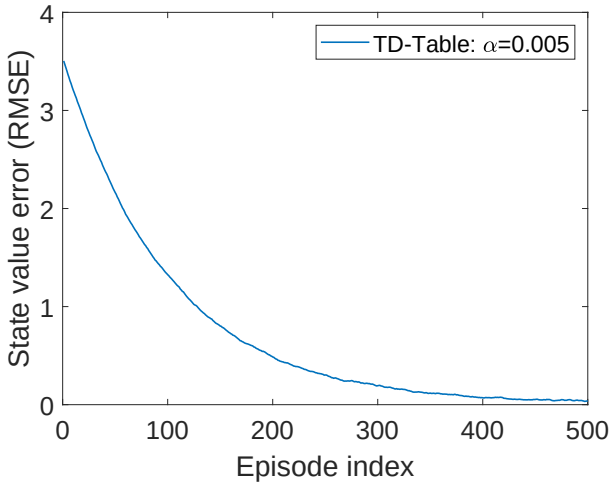
- 按给定策略生成了500 个回合。
- 每个回合有500 步，起始状态-动作对从均匀分布中随机选取。

示例

TD-Table:

- 作为对比，表格TD 算法（此处称为TD-Table）的结果：





示例

TD-Linear:

- 如何应用TD-Linear 算法?
  - 特征向量选择:

$$\phi(s) = \begin{bmatrix} 1 \\ x \\ y \end{bmatrix} \in \mathbb{R}^3.$$

  - 此时，近似状态值为:

$$\hat{v}(s, w) = \phi^T(s)w = [1, x, y] \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = w_1 + w_2 x + w_3 y.$$

注: $\phi(s)$ 也可定义为$\phi(s) = [x, y, 1]^T$，元素顺序不影响结果。

示例

TD-Linear:

- 如何应用TD-Linear 算法?
  - 特征向量选择:

$$\phi(s) = \begin{bmatrix} 1 \\ x \\ y \end{bmatrix} \in \mathbb{R}^3.$$

  - 此时，近似状态值为:

$$\hat{v}(s, w) = \phi^T(s)w = [1, x, y] \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = w_1 + w_2 x + w_3 y.$$

注: $\phi(s)$ 也可定义为$\phi(s) = [x, y, 1]^T$，元素顺序不影响结果。

示例

TD-Linear:

- 如何应用TD-Linear 算法?
  - 特征向量选择:

$$\phi(s) = \begin{bmatrix} 1 \\ x \\ y \end{bmatrix} \in \mathbb{R}^3.$$

  - 此时，近似状态值为:

$$\hat{v}(s, w) = \phi^T(s)w = [1, x, y] \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = w_1 + w_2 x + w_3 y.$$

注: $\phi(s)$ 也可定义为$\phi(s) = [x, y, 1]^T$，元素顺序不影响结果。

示例

TD-Linear:

- 如何应用TD-Linear 算法?
  - 特征向量选择:

$$\phi(s) = \begin{bmatrix} 1 \\ x \\ y \end{bmatrix} \in \mathbb{R}^3.$$

  - 此时，近似状态值为:

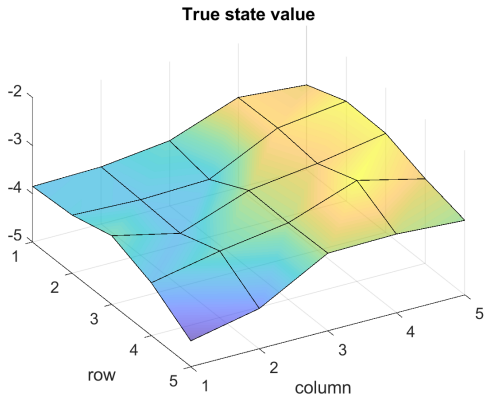
$$\hat{v}(s, w) = \phi^T(s)w = [1, x, y] \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = w_1 + w_2 x + w_3 y.$$

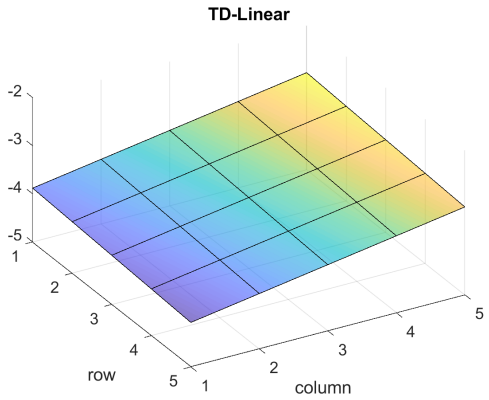
注：$\phi(s)$ 也可定义为$\phi(s) = [x, y, 1]^T$，元素顺序不影响结果。

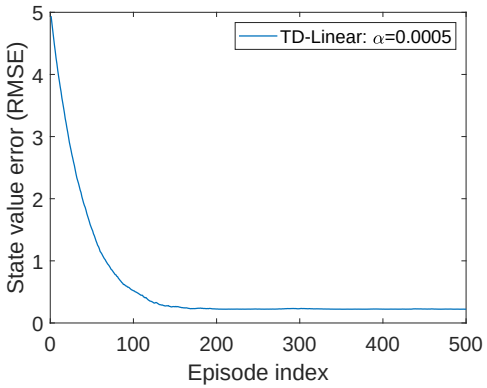
示例

TD-Linear:

- TD-Linear 算法的结果:





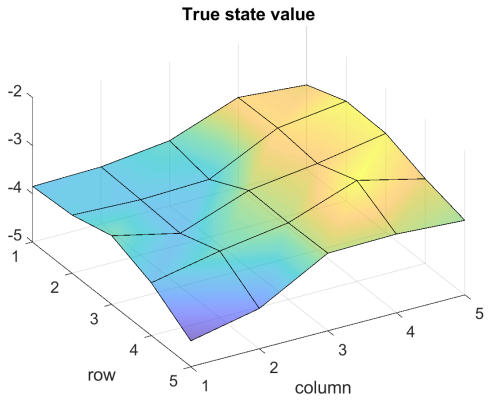


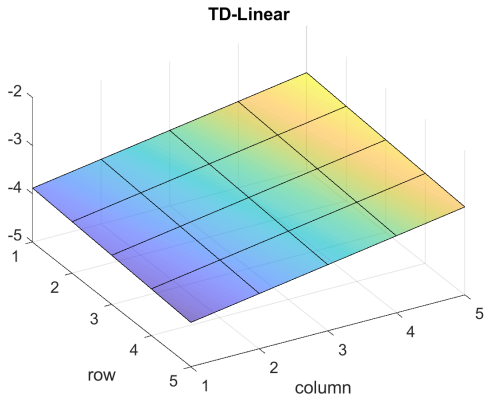
- 趋势正确，但由于近似能力有限存在误差！
- 我们试图用平面来近似一个非平面曲面！

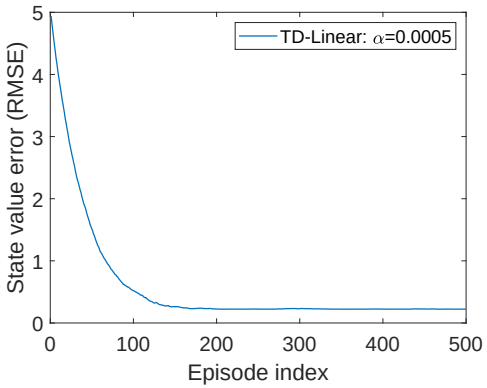
示例

TD-Linear:

- TD-Linear 算法的结果:







- 趋势正确，但由于近似能力有限存在误差！
- 我们试图用平面来近似一个非平面曲面！

示例

为增强近似能力，可以使用高阶特征向量，从而增加参数数量。

- 例如，可以考虑：

$$\phi(s) = [1, x, y, x^2, y^2, xy]^T \in \mathbb{R}^6.$$

  此时，

$$\hat{v}(s, w) = \phi^T(s)w = w_1 + w_2 x + w_3 y + w_4 x^2 + w_5 y^2 + w_6 xy$$

  对应一个二次曲面。

- 还可以进一步增加特征向量的维度：

$$\phi(s) = [1, x, y, x^2, y^2, xy, x^3, y^3, x^2y, xy^2]^T \in \mathbb{R}^{10}.$$

示例

为增强近似能力，可以使用高阶特征向量，从而增加参数数量。

- 例如，可以考虑：

$$\phi(s) = [1, x, y, x^2, y^2, xy]^T \in \mathbb{R}^6.$$

此时，

$$\hat{v}(s, w) = \phi^T(s)w = w_1 + w_2 x + w_3 y + w_4 x^2 + w_5 y^2 + w_6 xy$$

对应一个二次曲面。

- 还可以进一步增加特征向量的维度：

$$\phi(s) = [1, x, y, x^2, y^2, xy, x^3, y^3, x^2 y, xy^2]^T \in \mathbb{R}^{10}.$$

示例

为增强近似能力，可以使用高阶特征向量，从而增加参数数量。

- 例如，可以考虑：

$$\phi(s) = [1, x, y, x^2, y^2, xy]^T \in \mathbb{R}^6.$$

  此时，

$$\hat{v}(s, w) = \phi^T(s)w = w_1 + w_2 x + w_3 y + w_4 x^2 + w_5 y^2 + w_6 xy$$

  对应一个二次曲面。

- 还可以进一步增加特征向量的维度：

$$\phi(s) = [1, x, y, x^2, y^2, xy, x^3, y^3, x^2y, xy^2]^T \in \mathbb{R}^{10}.$$

示例

为增强近似能力，可以使用高阶特征向量，从而增加参数数量。

- 例如，可以考虑：

$$\phi(s) = [1, x, y, x^2, y^2, xy]^T \in \mathbb{R}^6.$$

  此时，

$$\hat{v}(s, w) = \phi^T(s)w = w_1 + w_2 x + w_3 y + w_4 x^2 + w_5 y^2 + w_6 xy$$

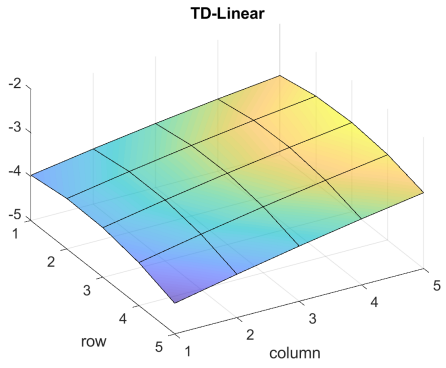
  对应一个二次曲面。

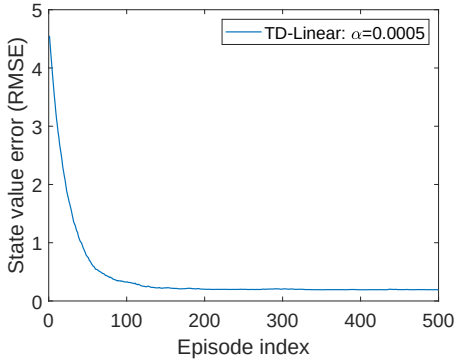
- 还可以进一步增加特征向量的维度：

$$\phi(s) = [1, x, y, x^2, y^2, xy, x^3, y^3, x^2y, xy^2]^T \in \mathbb{R}^{10}.$$

示例

使用高阶特征向量的TD-Linear 结果：



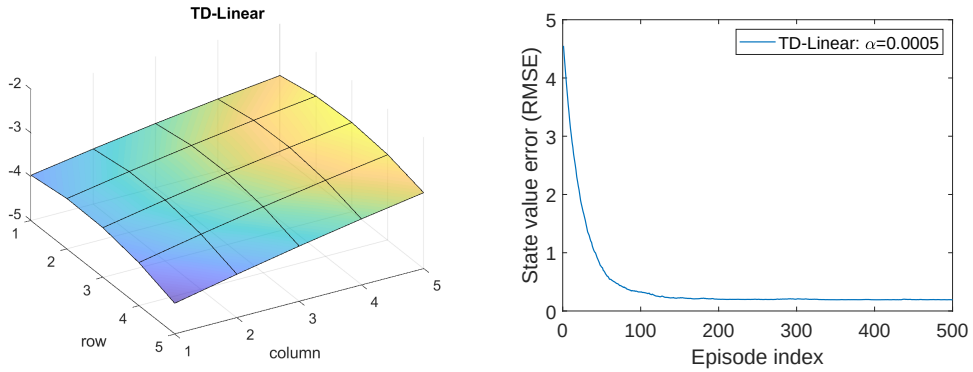


上图：$\phi(s) \in \mathbb{R}^6$

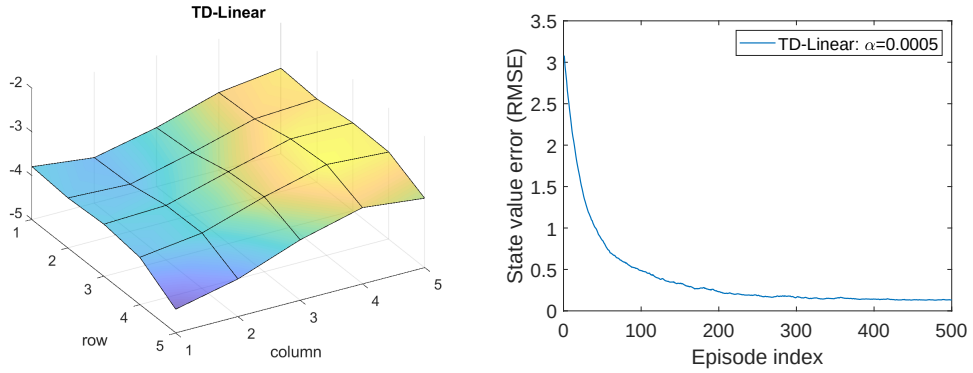
更多示例和特征向量见教材。

示例

使用高阶特征向量的TD-Linear 结果：



上图：$\phi(s) \in \mathbb{R}^6$

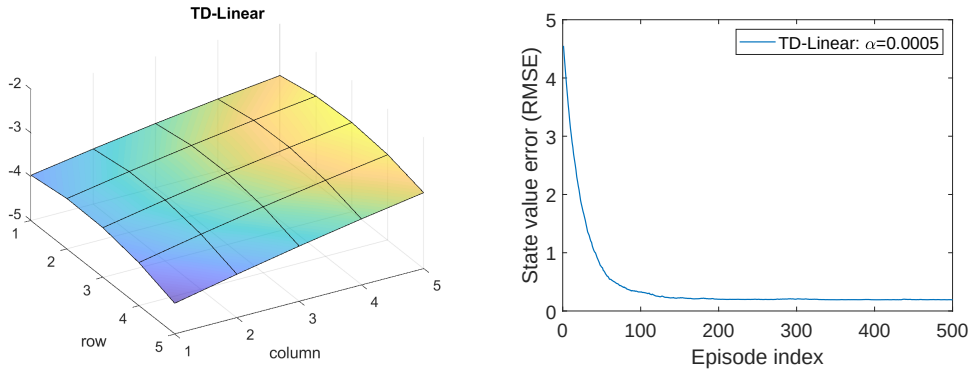


上图：$\phi(s) \in \mathbb{R}^{10}$

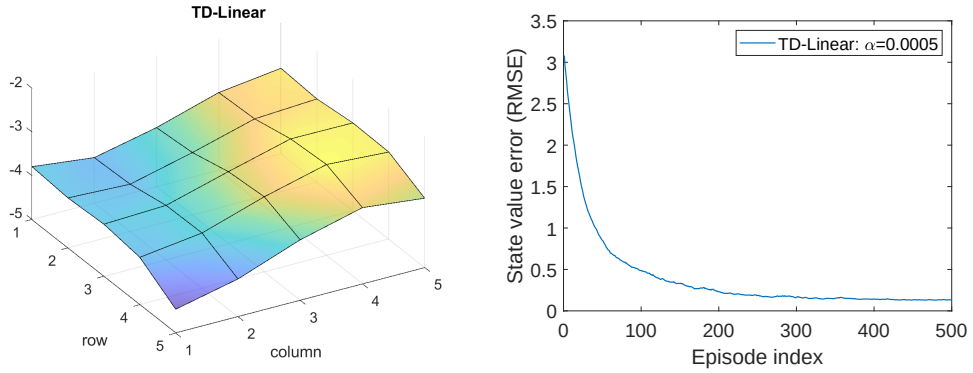
更多示例和特征向量见教材。

示例

使用高阶特征向量的TD-Linear 结果：



上图：$\phi(s) \in \mathbb{R}^6$



上图：$\phi(s) \in \mathbb{R}^{10}$

更多示例和特征向量见教材。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

故事总结

至此，我们完成了带值函数近似的TD 学习的完整故事。

1) 故事从目标函数出发：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

目标函数表明这是一个策略评估问题。

2) 梯度下降算法为：

$$w_{t+1} = w_t + \alpha_t(v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

3) 算法中未知的真实值函数被近似值替换，得到：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t)$$

尽管这一故事有助于理解基本思想，但它在数学上并不严格。

故事总结

至此，我们完成了带值函数近似的TD 学习的完整故事。

1) 故事从目标函数出发：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

目标函数表明这是一个策略评估问题。

2) 梯度下降算法为：

$$w_{t+1} = w_t + \alpha_t(v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

3) 算法中未知的真实值函数被近似值替换，得到：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t)$$

尽管这一故事有助于理解基本思想，但它在数学上并不严格。

故事总结

至此，我们完成了带值函数近似的TD 学习的完整故事。

1) 故事从目标函数出发：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

目标函数表明这是一个策略评估问题。

2) 梯度下降算法为：

$$w_{t+1} = w_t + \alpha_t(v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

3) 算法中未知的真实值函数被近似值替换，得到：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t)$$

尽管这一故事有助于理解基本思想，但它在数学上并不严格。

故事总结

至此，我们完成了带值函数近似的TD 学习的完整故事。

1) 故事从目标函数出发：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

目标函数表明这是一个策略评估问题。

2) 梯度下降算法为：

$$w_{t+1} = w_t + \alpha_t(v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

3) 算法中未知的真实值函数被近似值替换，得到：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t)$$

尽管这一故事有助于理解基本思想，但它在数学上并不严格。

故事总结

至此，我们完成了带值函数近似的TD 学习的完整故事。

1) 故事从目标函数出发：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

目标函数表明这是一个策略评估问题。

2) 梯度下降算法为：

$$w_{t+1} = w_t + \alpha_t(v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

3) 算法中未知的真实值函数被近似值替换，得到：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t)$$

尽管这一故事有助于理解基本思想，但它在数学上并不严格。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

理论分析（选读）

- 算法:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t) \right] \nabla_w \hat{v}(s_t, w_t)$$

并不最小化如下目标函数:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

理论分析（选读）

不同的目标函数:

- 目标函数一：真实值误差

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \|\hat{v}(w) - v_\pi\|_D^2$$

- 目标函数二：**Bellman** 误差

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \doteq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2,$$

其中$T_\pi(x) \doteq r_\pi + \gamma P_\pi x$

- 目标函数三：**投影Bellman** 误差

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2,$$

其中M 为投影矩阵。

- TD-Linear 算法最小化投影Bellman 误差。

更多细节此处略去，感兴趣的读者可参见教材。

理论分析（选读）

不同的目标函数:

- **目标函数一：真实值误差**

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \|\hat{v}(w) - v_\pi\|_D^2$$

- **目标函数二：Bellman 误差**

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \doteq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2,$$

其中$T_\pi(x) \doteq r_\pi + \gamma P_\pi x$

- **目标函数三：投影Bellman 误差**

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2,$$

其中M 为投影矩阵。

- TD-Linear 算法最小化投影Bellman 误差。

更多细节此处略去，感兴趣的读者可参见教材。

理论分析（选读）

不同的目标函数:

- **目标函数一：真实值误差**

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \|\hat{v}(w) - v_\pi\|_D^2$$

- **目标函数二：Bellman 误差**

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \doteq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2,$$

其中$T_\pi(x) \doteq r_\pi + \gamma P_\pi x$

- **目标函数三：投影Bellman 误差**

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2,$$

其中M 为投影矩阵。

- TD-Linear 算法最小化投影Bellman 误差。

更多细节此处略去，感兴趣的读者可参见教材。

理论分析（选读）

不同的目标函数:

- **目标函数一：真实值误差**

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \|\hat{v}(w) - v_\pi\|_D^2$$

- **目标函数二：Bellman 误差**

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \doteq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2,$$

其中$T_\pi(x) \doteq r_\pi + \gamma P_\pi x$

- **目标函数三：投影Bellman 误差**

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2,$$

其中M 为投影矩阵。

- TD-Linear 算法最小化投影Bellman 误差。

更多细节此处略去，感兴趣的读者可参见教材。

理论分析（选读）

不同的目标函数:

- **目标函数一：真实值误差**

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \|\hat{v}(w) - v_\pi\|_D^2$$

- **目标函数二：Bellman 误差**

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \doteq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2,$$

其中$T_\pi(x) \doteq r_\pi + \gamma P_\pi x$

- **目标函数三：投影Bellman 误差**

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2,$$

其中M 为投影矩阵。

- TD-Linear 算法最小化投影Bellman 误差。

更多细节此处略去，感兴趣的读者可参见教材。

理论分析（选读）

不同的目标函数:

- **目标函数一：真实值误差**

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \|\hat{v}(w) - v_\pi\|_D^2$$

- **目标函数二：Bellman 误差**

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \doteq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2,$$

其中$T_\pi(x) \doteq r_\pi + \gamma P_\pi x$

- **目标函数三：投影Bellman 误差**

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2,$$

其中M 为投影矩阵。

- TD-Linear 算法最小化投影Bellman 误差。

更多细节此处略去，感兴趣的读者可参见教材。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

带函数近似的Sarsa

目前仅考虑了状态值估计，即：

$$\hat{v}(s) \approx v_\pi(s), \quad s \in \mathcal{S}$$

为搜索最优策略，需要估计动作值。

带值函数近似的Sarsa 算法为：

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t).$$

与本讲前面介绍的算法相同，只是将$\hat{v}$ 替换为$\hat{q}$。

带函数近似的Sarsa

目前仅考虑了状态值估计，即：

$$\hat{v}(s) \approx v_\pi(s), \quad s \in \mathcal{S}$$

为搜索最优策略，需要估计动作值。

带值函数近似的Sarsa 算法为：

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t).$$

与本讲前面介绍的算法相同，只是将$\hat{v}$ 替换为$\hat{q}$。

带函数近似的Sarsa

目前仅考虑了状态值估计，即：

$$\hat{v}(s) \approx v_\pi(s), \quad s \in \mathcal{S}$$

为搜索最优策略，需要估计动作值。

带值函数近似的Sarsa 算法为：

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t).$$

与本讲前面介绍的算法相同，只是将$\hat{v}$ 替换为$\hat{q}$。

带函数近似的Sarsa

目前仅考虑了状态值估计，即：

$$\hat{v}(s) \approx v_\pi(s), \quad s \in \mathcal{S}$$

为搜索最优策略，需要估计动作值。

带值函数近似的Sarsa 算法为：

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t).$$

与本讲前面介绍的算法相同，只是将$\hat{v}$ 替换为$\hat{q}$。

带函数近似的Sarsa

为搜索最优策略，可将策略评估与策略改进结合。

带函数近似的Sarsa

为搜索最优策略，可将策略评估与策略改进结合。

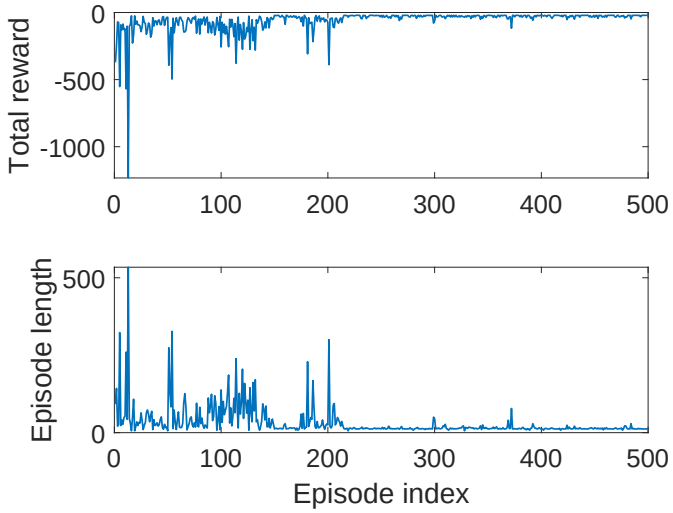
伪代码：带函数近似的Sarsa

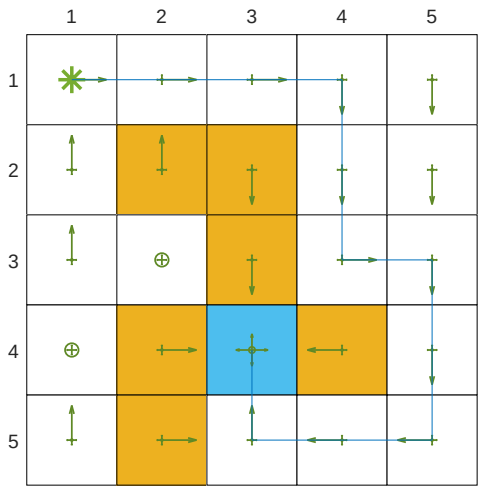
初始化： 初始参数w_0，初始策略π_0。对所有t，$\alpha_t = \alpha > 0$，$\epsilon \in (0,1)$。
目标： 学习从初始状态s_0 引导智能体到达目标状态的最优策略。
对每个回合，执行：
在s_0 处按$\pi_0(s_0)$ 生成a_0
若s_t（$t = 0, 1, 2, \ldots$）不是目标状态，执行：
给定(s_t, a_t)，采集经验样本$(r_{t+1}, s_{t+1}, a_{t+1})$：与环境交互生成$r_{t+1}, s_{t+1}$；
按$\pi_t(s_{t+1})$ 生成a_{t+1}。
更新Q 值（更新参数）：
$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t)$$
更新策略：
$\pi_{t+1}(a|s_t) = 1 - \frac{\varepsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)| - 1)$，若$a = \arg\max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1})$
$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|}$，否则
$s_t \leftarrow s_{t+1}$，$a_t \leftarrow a_{t+1}$

带函数近似的Sarsa

示例：

- 带线性函数近似的Sarsa：$\hat{q}(s,a,w) = \phi^T(s,a)w$
- $\gamma = 0.9$，$\epsilon = 0.1$，$r_{\text{boundary}} = r_{\text{forbidden}} = -10$，$r_{\text{target}} = 1$，$\alpha = 0.001$。





详细内容请参见教材。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

带函数近似的Q 学习

与Sarsa 类似，表格Q 学习也可扩展至值函数近似的情形。

Q 值更新规则为：

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t),$$

与Sarsa 的区别仅在于将$\hat{q}(s_{t+1}, a_{t+1}, w_t)$ 替换为$\max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t)$。

带函数近似的Q 学习

与Sarsa 类似，表格Q 学习也可扩展至值函数近似的情形。

Q 值更新规则为：

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t),$$

与Sarsa 的区别仅在于将$\hat{q}(s_{t+1}, a_{t+1}, w_t)$ 替换为$\max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t)$。

带函数近似的Q 学习

伪代码：带函数近似的Q 学习（同策略版本）

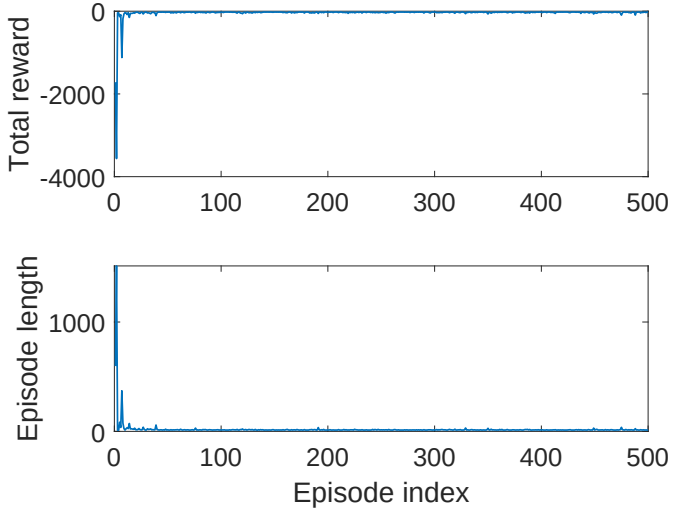
初始化： 初始参数w_0，初始策略π_0。对所有t，$\alpha_t = \alpha > 0$，$\epsilon \in (0,1)$。
目标： 学习从初始状态s_0 引导智能体到达目标状态的最优路径。

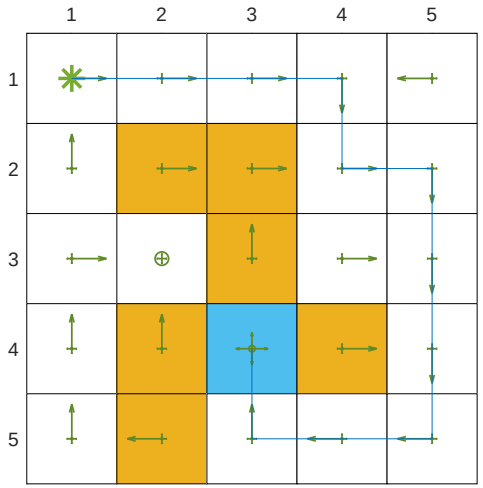
对每个回合，执行：
若s_t（$t = 0, 1, 2, \ldots$）不是目标状态，执行：
给定s_t，采集经验样本(a_t, r_{t+1}, s_{t+1})：按$\pi_t(s_t)$ 生成a_t；与环境交互生成r_{t+1}, s_{t+1}。
更新值（更新参数）：
$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$
更新策略：
$\pi_{t+1}(a|s_t) = 1 - \frac{\varepsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)| - 1)$，若$a = \arg\max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1})$
$\pi_{t+1}(a|s_t) = \frac{\varepsilon}{|\mathcal{A}(s_t)|}$，否则

带函数近似的Q 学习

示例：

- 带线性函数近似的Q 学习：$\hat{q}(s,a,w) = \phi^T(s,a)w$
- $\gamma = 0.9$，$\epsilon = 0.1$，$r_{\text{boundary}} = r_{\text{forbidden}} = -10$，$r_{\text{target}} = 1$，$\alpha = 0.001$。





大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

深度Q 学习

深度Q 学习或深度Q 网络（DQN）：

- 最早且最成功的将深度神经网络引入强化学习的算法之一。
- 神经网络的作用是充当非线性函数近似器。
- 与以下算法不同:

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t)$$

区别在于训练网络的方式。

深度Q 学习

深度Q 学习或深度Q 网络（DQN）：

- 最早且最成功的将深度神经网络引入强化学习的算法之一。
- 神经网络的作用是充当非线性函数近似器。
- 与以下算法不同:

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t)$$

区别在于训练网络的方式。

深度Q 学习

深度Q 学习或深度Q 网络（DQN）：

- 最早且最成功的将深度神经网络引入强化学习的算法之一。
- 神经网络的作用是充当非线性函数近似器。
- 与以下算法不同：

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t)$$

区别在于训练网络的方式。

深度Q 学习

深度Q 学习或深度Q 网络（DQN）：

- 最早且最成功的将深度神经网络引入强化学习的算法之一。
- 神经网络的作用是充当非线性函数近似器。
- 与以下算法不同:

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t)$$

区别在于训练网络的方式。

深度Q 学习

深度Q 学习旨在最小化如下目标函数/损失函数:

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t)$$

$$\Downarrow$$

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w) \right)^2 \right]$$

其中(S, A, R, S') 为随机变量。

深度Q 学习

如何最小化目标函数？梯度下降！

- 如何计算目标函数的梯度？有些棘手！
- 因为在目标函数：

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

中，参数w 不仅出现在$\hat{q}(S, A, w)$ 中，还出现在：

$$y \doteq R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w)$$

- 由于最优a 依赖于w，

$$\nabla_w y \neq \gamma \max_{a \in \mathcal{A}(S')} \nabla_w \hat{q}(S', a, w)$$

- 为解决这一问题，可以在计算梯度时假设y 中的w 固定（至少暂时固定）。

深度Q 学习

如何最小化目标函数？梯度下降！

- 如何计算目标函数的梯度？有些棘手！
- 因为在目标函数：

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

中，参数w 不仅出现在$\hat{q}(S, A, w)$ 中，还出现在：

$$y \doteq R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w)$$

- 由于最优a 依赖于w，

$$\nabla_w y \neq \gamma \max_{a \in \mathcal{A}(S')} \nabla_w \hat{q}(S', a, w)$$

- 为解决这一问题，可以在计算梯度时假设y 中的w 固定（至少暂时固定）。

深度Q 学习

如何最小化目标函数? 梯度下降!

- 如何计算目标函数的梯度? 有些棘手!
- 因为在目标函数:

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

中，参数w 不仅出现在$\hat{q}(S, A, w)$ 中，还出现在:

$$y \doteq R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w)$$

- 由于最优a 依赖于w,

$$\nabla_w y \neq \gamma \max_{a \in \mathcal{A}(S')} \nabla_w \hat{q}(S', a, w)$$

- 为解决这一问题，可以在计算梯度时假设y 中的w 固定（至少暂时固定）。

深度Q 学习

如何最小化目标函数? 梯度下降!

- 如何计算目标函数的梯度? 有些棘手!
- 因为在目标函数:

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

中，参数w 不仅出现在$\hat{q}(S, A, w)$ 中，还出现在:

$$y \doteq R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w)$$

- 由于最优a 依赖于w,

$$\nabla_w y \neq \gamma \max_{a \in \mathcal{A}(S')} \nabla_w \hat{q}(S', a, w)$$

- 为解决这一问题，可以在计算梯度时假设y 中的w 固定（至少暂时固定）。

深度Q 学习

如何最小化目标函数? 梯度下降!

- 如何计算目标函数的梯度? 有些棘手!
- 因为在目标函数:

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

中，参数w 不仅出现在$\hat{q}(S, A, w)$ 中，还出现在:

$$y \doteq R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w)$$

- 由于最优a 依赖于w,

$$\nabla_w y \neq \gamma \max_{a \in \mathcal{A}(S')} \nabla_w \hat{q}(S', a, w)$$

- 为解决这一问题，可以在计算梯度时假设y 中的w 固定（至少暂时固定）。

深度Q 学习

为此，引入两个网络：

- 一个是表示$\hat{q}(s,a,w)$ 的主网络
- 另一个是目标网络 $\hat{q}(s,a,w_T)$。

此时目标函数退化为：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right],$$

其中w_T 为目标网络参数。
当w_T 固定时，J 的梯度可方便地得到：

$$\nabla_w J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right) \nabla_w \hat{q}(S, A, w)\right].$$

- 深度Q 学习的基本思想是使用梯度下降算法最小化目标函数。
- 然而，该优化过程涉及若干重要技巧，值得特别关注。

深度Q 学习

为此，引入两个网络：

- 一个是表示$\hat{q}(s,a,w)$ 的主网络
- 另一个是目标网络 $\hat{q}(s,a,w_T)$。

此时目标函数退化为：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right],$$

其中w_T 为目标网络参数。
当w_T 固定时，J 的梯度可方便地得到：

$$\nabla_w J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right) \nabla_w \hat{q}(S, A, w)\right].$$

- 深度Q 学习的基本思想是使用梯度下降算法最小化目标函数。
- 然而，该优化过程涉及若干重要技巧，值得特别关注。

深度Q 学习

为此，引入两个网络：

- 一个是表示$\hat{q}(s,a,w)$ 的主网络
- 另一个是目标网络 $\hat{q}(s,a,w_T)$。

此时目标函数退化为：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right],$$

其中w_T 为目标网络参数。

当w_T 固定时，J 的梯度可方便地得到：

$$\nabla_w J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right) \nabla_w \hat{q}(S, A, w)\right].$$

- 深度Q 学习的基本思想是使用梯度下降算法最小化目标函数。
- 然而，该优化过程涉及若干重要技巧，值得特别关注。

深度Q 学习

为此，引入两个网络：

- 一个是表示$\hat{q}(s,a,w)$ 的主网络
- 另一个是目标网络 $\hat{q}(s,a,w_T)$。

此时目标函数退化为：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right],$$

其中w_T 为目标网络参数。
当w_T 固定时，J 的梯度可方便地得到：

$$\nabla_w J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right) \nabla_w \hat{q}(S, A, w)\right].$$

- 深度Q 学习的基本思想是使用梯度下降算法最小化目标函数。
- 然而，该优化过程涉及若干重要技巧，值得特别关注。

深度Q 学习

为此，引入两个网络：

- 一个是表示$\hat{q}(s,a,w)$ 的主网络
- 另一个是目标网络 $\hat{q}(s,a,w_T)$。

此时目标函数退化为：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right],$$

其中w_T 为目标网络参数。
当w_T 固定时，J 的梯度可方便地得到：

$$\nabla_w J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right) \nabla_w \hat{q}(S, A, w)\right].$$

- 深度Q 学习的基本思想是使用梯度下降算法最小化目标函数。
- 然而，该优化过程涉及若干重要技巧，值得特别关注。

深度Q 学习

为此，引入两个网络：

- 一个是表示$\hat{q}(s,a,w)$ 的主网络
- 另一个是目标网络 $\hat{q}(s,a,w_T)$。

此时目标函数退化为：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right],$$

其中w_T 为目标网络参数。
当w_T 固定时，J 的梯度可方便地得到：

$$\nabla_w J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right) \nabla_w \hat{q}(S, A, w)\right].$$

- 深度Q 学习的基本思想是使用梯度下降算法最小化目标函数。
- 然而，该优化过程涉及若干重要技巧，值得特别关注。

深度Q 学习——两个网络

技巧一： 两个网络：主网络与目标网络。

为何使用?

- 数学原因已在计算梯度时说明。

实现细节:

- 设w 和w_T 分别为主网络和目标网络的参数，初始时设为相同。
- 每次迭代时，从经验回放缓冲区（稍后说明）中抽取一个小批量样本$\{(s, a, r, s')\}$。
- 对每个(s, a, r, s')，计算目标输出:

$$y_T \doteq r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$$

从而得到一个小批量数据:

$$\{(s, a, y_T)\}$$

- 用$\{(s, a, y_T)\}$ 训练网络，最小化$(y_T - \hat{q}(s, a, w))^2$。

深度Q 学习——两个网络

技巧一： 两个网络：主网络与目标网络。

为何使用?

- 数学原因已在计算梯度时说明。

实现细节：

- 设w 和w_T 分别为主网络和目标网络的参数，初始时设为相同。
- 每次迭代时，从经验回放缓冲区（稍后说明）中抽取一个小批量样本$\{(s, a, r, s')\}$。
- 对每个(s, a, r, s')，计算目标输出：

$$y_T \doteq r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$$

  从而得到一个小批量数据：

$$\{(s, a, y_T)\}$$

- 用$\{(s, a, y_T)\}$ 训练网络，最小化$(y_T - \hat{q}(s, a, w))^2$。

深度Q 学习——两个网络

技巧一： 两个网络：主网络与目标网络。

为何使用?

- 数学原因已在计算梯度时说明。

实现细节：

- 设w 和w_T 分别为主网络和目标网络的参数，初始时设为相同。
- 每次迭代时，从经验回放缓冲区（稍后说明）中抽取一个小批量样本$\{(s, a, r, s')\}$。
- 对每个(s, a, r, s')，计算目标输出：

$$y_T \doteq r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$$

  从而得到一个小批量数据：

$$\{(s, a, y_T)\}$$

- 用$\{(s, a, y_T)\}$ 训练网络，最小化$(y_T - \hat{q}(s, a, w))^2$。

深度Q 学习——两个网络

技巧一： 两个网络：主网络与目标网络。

为何使用?

- 数学原因已在计算梯度时说明。

实现细节：

- 设w 和w_T 分别为主网络和目标网络的参数，初始时设为相同。
- 每次迭代时，从经验回放缓冲区（稍后说明）中抽取一个小批量样本$\{(s, a, r, s')\}$。
- 对每个(s, a, r, s')，计算目标输出：

$$y_T \doteq r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$$

  从而得到一个小批量数据：

$$\{(s, a, y_T)\}$$

- 用$\{(s, a, y_T)\}$ 训练网络，最小化$(y_T - \hat{q}(s, a, w))^2$。

深度Q 学习——两个网络

技巧一： 两个网络：主网络与目标网络。

为何使用?

- 数学原因已在计算梯度时说明。

实现细节：

- 设w 和w_T 分别为主网络和目标网络的参数，初始时设为相同。
- 每次迭代时，从经验回放缓冲区（稍后说明）中抽取一个小批量样本$\{(s, a, r, s')\}$。
- 对每个(s, a, r, s')，计算目标输出：

$$y_T \doteq r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$$

从而得到一个小批量数据：

$$\{(s, a, y_T)\}$$

- 用$\{(s, a, y_T)\}$ 训练网络，最小化$(y_T - \hat{q}(s, a, w))^2$。

深度Q 学习——两个网络

技巧一： 两个网络：主网络与目标网络。

为何使用?

- 数学原因已在计算梯度时说明。

实现细节：

- 设w 和w_T 分别为主网络和目标网络的参数，初始时设为相同。
- 每次迭代时，从经验回放缓冲区（稍后说明）中抽取一个小批量样本$\{(s, a, r, s')\}$。
- 对每个(s, a, r, s')，计算目标输出：

$$y_T \doteq r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$$

从而得到一个小批量数据：

$$\{(s, a, y_T)\}$$

- 用$\{(s, a, y_T)\}$ 训练网络，最小化$(y_T - \hat{q}(s, a, w))^2$。

技巧二： 经验回放

问题： 什么是经验回放?

回答：

- 在采集一些经验样本后，不按采集顺序使用这些样本。
- 而是将它们存储在一个集合中，称为回放缓冲区$\mathcal{B} \doteq \{(s, a, r, s')\}$。
- 每次训练神经网络时，从回放缓冲区随机抽取一个小批量样本。
- 样本的抽取，即经验回放，应遵循均匀分布。

技巧二： 经验回放

问题： 什么是经验回放?

回答：

- 在采集一些经验样本后，不按采集顺序使用这些样本。
- 而是将它们存储在一个集合中，称为回放缓冲区$\mathcal{B} \doteq \{(s, a, r, s')\}$。
- 每次训练神经网络时，从回放缓冲区随机抽取一个小批量样本。
- 样本的抽取，即经验回放，应遵循均匀分布。

技巧二： 经验回放

问题： 什么是经验回放?

回答：

- 在采集一些经验样本后，不按采集顺序使用这些样本。
- 而是将它们存储在一个集合中，称为回放缓冲区$\mathcal{B} \doteq \{(s, a, r, s')\}$。
- 每次训练神经网络时，从回放缓冲区随机抽取一个小批量样本。
- 样本的抽取，即经验回放，应遵循均匀分布。

技巧二： 经验回放

问题： 什么是经验回放?

回答：

- 在采集一些经验样本后，不按采集顺序使用这些样本。
- 而是将它们存储在一个集合中，称为回放缓冲区$\mathcal{B} \doteq \{(s, a, r, s')\}$。
- 每次训练神经网络时，从回放缓冲区随机抽取一个小批量样本。
- 样本的抽取，即经验回放，应遵循均匀分布。

深度Q 学习——经验回放

技巧二： 经验回放

问题： 什么是经验回放?

回答：

- 在采集一些经验样本后，不按采集顺序使用这些样本。
- 而是将它们存储在一个集合中，称为回放缓冲区$\mathcal{B} \doteq \{(s, a, r, s')\}$。
- 每次训练神经网络时，从回放缓冲区随机抽取一个小批量样本。
- 样本的抽取，即经验回放，应遵循均匀分布。

技巧二： 经验回放

问题： 什么是经验回放?

回答：

- 在采集一些经验样本后，不按采集顺序使用这些样本。
- 而是将它们存储在一个集合中，称为回放缓冲区$\mathcal{B} \doteq \{(s, a, r, s')\}$。
- 每次训练神经网络时，从回放缓冲区随机抽取一个小批量样本。
- 样本的抽取，即经验回放，应遵循均匀分布。

深度Q 学习——经验回放

问题： 为什么深度Q 学习中经验回放是必要的？为什么回放必须遵循均匀分布？

回答： 答案在于目标函数：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

- $R \sim p(R|S, A)$，$S' \sim p(S'|S, A)$：R 和S' 由系统模型决定。
- $(S, A) \sim d$：(S, A) 是索引。
- 状态-动作对(S, A) 的分布假设为均匀分布。
  - 为什么用均匀分布？因为没有先验知识。
  - 能否像之前一样用稳态分布？不能，因为没有给定策略。

深度Q 学习——经验回放

问题： 为什么深度Q 学习中经验回放是必要的？为什么回放必须遵循均匀分布？

回答： 答案在于目标函数：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

- $R \sim p(R|S, A)$，$S' \sim p(S'|S, A)$：R 和S' 由系统模型决定。
- $(S, A) \sim d$：(S, A) 是索引。
- 状态-动作对(S, A) 的分布假设为均匀分布。
  - 为什么用均匀分布？因为没有先验知识。
  - 能否像之前一样用稳态分布？不能，因为没有给定策略。

深度Q 学习——经验回放

问题： 为什么深度Q 学习中经验回放是必要的？为什么回放必须遵循均匀分布？

回答： 答案在于目标函数：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

- $R \sim p(R|S, A)$，$S' \sim p(S'|S, A)$：R 和S' 由系统模型决定。
- $(S, A) \sim d$：(S, A) 是索引。
- 状态-动作对(S, A) 的分布假设为均匀分布。
  - 为什么用均匀分布？因为没有先验知识。
  - 能否像之前一样用稳态分布？不能，因为没有给定策略。

深度Q 学习——经验回放

问题： 为什么深度Q 学习中经验回放是必要的？为什么回放必须遵循均匀分布？

回答： 答案在于目标函数：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

- $R \sim p(R|S, A)$，$S' \sim p(S'|S, A)$：R 和S' 由系统模型决定。
- $(S, A) \sim d$：(S, A) 是索引。
- 状态-动作对(S, A) 的分布假设为均匀分布。
  - 为什么用均匀分布？因为没有先验知识。
  - 能否像之前一样用稳态分布？不能，因为没有给定策略。

深度Q 学习——经验回放

问题： 为什么深度Q 学习中经验回放是必要的？为什么回放必须遵循均匀分布？

回答： 答案在于目标函数：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

- $R \sim p(R|S, A)$，$S' \sim p(S'|S, A)$：R 和S' 由系统模型决定。
- $(S, A) \sim d$：(S, A) 是索引。
- 状态-动作对(S, A) 的分布假设为均匀分布。
  - 为什么用均匀分布？因为没有先验知识。
  - 能否像之前一样用稳态分布？不能，因为没有给定策略。

深度Q 学习——经验回放

问题： 为什么深度Q 学习中经验回放是必要的？为什么回放必须遵循均匀分布？

回答： 答案在于目标函数：

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$$

- $R \sim p(R|S, A)$，$S' \sim p(S'|S, A)$：R 和S' 由系统模型决定。
- $(S, A) \sim d$：(S, A) 是索引。
- 状态-动作对(S, A) 的分布假设为均匀分布。
  - 为什么用均匀分布？因为没有先验知识。
  - 能否像之前一样用稳态分布？不能，因为没有给定策略。

回答（续）：

- 然而，样本并非均匀采集，因为它们是由某些策略依次生成的。
- 为打破相邻样本之间的相关性，可以通过经验回放技术从回放缓冲区中均匀抽取样本。
- 这是经验回放必要且必须均匀采样的数学原因。

回答（续）：

- 然而，样本并非均匀采集，因为它们是由某些策略依次生成的。
- 为打破相邻样本之间的相关性，可以通过经验回放技术从回放缓冲区中均匀抽取样本。
- 这是经验回放必要且必须均匀采样的数学原因。

回答（续）：

- 然而，样本并非均匀采集，因为它们是由某些策略依次生成的。
- 为打破相邻样本之间的相关性，可以通过经验回放技术从回放缓冲区中均匀抽取样本。
- 这是经验回放必要且必须均匀采样的数学原因。

深度Q 学习——经验回放

回顾表格情形：

- 问：为什么表格Q 学习不需要经验回放?
  - 答：因为它不需要S 或A 的任何分布。
- 问：为什么深度Q 学习涉及分布?
  - 答：因为需要定义一个标量目标函数$J(w) = \mathbb{E}[*]$，其中$\mathbb{E}$ 对所有(S, A) 取期望。
  - 表格情形旨在对所有(s, a) 求解方程组（Bellman 最优方程），而深度情形旨在优化一个标量目标函数。
- 问：可以在表格Q 学习中使用经验回放吗?
  - 答：可以，而且样本效率更高（为什么？）

深度Q 学习——经验回放

回顾表格情形：

- 问：为什么表格Q 学习不需要经验回放?
  - 答：因为它不需要S 或A 的任何分布。
- 问：为什么深度Q 学习涉及分布?
  - 答：因为需要定义一个标量目标函数$J(w) = \mathbb{E}[*]$，其中$\mathbb{E}$ 对所有(S, A) 取期望。
  - 表格情形旨在对所有(s, a) 求解方程组（Bellman 最优方程），而深度情形旨在优化一个标量目标函数。
- 问：可以在表格Q 学习中使用经验回放吗?
  - 答：可以，而且样本效率更高（为什么？）

深度Q 学习——经验回放

回顾表格情形：

- 问：为什么表格Q 学习不需要经验回放?
  - 答：因为它不需要S 或A 的任何分布。
- 问：为什么深度Q 学习涉及分布?
  - 答：因为需要定义一个标量目标函数$J(w) = \mathbb{E}[*]$，其中$\mathbb{E}$ 对所有(S, A) 取期望。
  - 表格情形旨在对所有(s, a) 求解方程组（Bellman 最优方程），而深度情形旨在优化一个标量目标函数。
- 问：可以在表格Q 学习中使用经验回放吗?
  - 答：可以，而且样本效率更高（为什么？）

深度Q 学习——经验回放

回顾表格情形：

- 问：为什么表格Q 学习不需要经验回放?
  - 答：因为它不需要S 或A 的任何分布。
- 问：为什么深度Q 学习涉及分布?
  - 答：因为需要定义一个标量目标函数$J(w) = \mathbb{E}[*]$，其中$\mathbb{E}$ 对所有(S, A) 取期望。
  - 表格情形旨在对所有(s, a) 求解方程组（Bellman 最优方程），而深度情形旨在优化一个标量目标函数。
- 问：可以在表格Q 学习中使用经验回放吗?
  - 答：可以，而且样本效率更高（为什么？）

深度Q 学习——经验回放

回顾表格情形：

- 问：为什么表格Q 学习不需要经验回放?
  - 答：因为它不需要S 或A 的任何分布。
- 问：为什么深度Q 学习涉及分布?
  - 答：因为需要定义一个标量目标函数$J(w) = \mathbb{E}[*]$，其中$\mathbb{E}$ 对所有(S, A) 取期望。
  - 表格情形旨在对所有(s, a) 求解方程组（Bellman 最优方程），而深度情形旨在优化一个标量目标函数。
- 问：可以在表格Q 学习中使用经验回放吗?
  - 答：可以，而且样本效率更高（为什么？）

深度Q 学习——经验回放

回顾表格情形：

- 问：为什么表格Q 学习不需要经验回放?
  - 答：因为它不需要S 或A 的任何分布。
- 问：为什么深度Q 学习涉及分布?
  - 答：因为需要定义一个标量目标函数$J(w) = \mathbb{E}[*]$，其中$\mathbb{E}$ 对所有(S, A) 取期望。
  - 表格情形旨在对所有(s, a) 求解方程组（Bellman 最优方程），而深度情形旨在优化一个标量目标函数。
- 问：可以在表格Q 学习中使用经验回放吗?
  - 答：可以，而且样本效率更高（为什么？）

深度Q 学习——经验回放

回顾表格情形：

- 问：为什么表格Q 学习不需要经验回放?
  - 答：因为它不需要S 或A 的任何分布。
- 问：为什么深度Q 学习涉及分布?
  - 答：因为需要定义一个标量目标函数$J(w) = \mathbb{E}[*]$，其中$\mathbb{E}$ 对所有(S, A) 取期望。
  - 表格情形旨在对所有(s, a) 求解方程组（Bellman 最优方程），而深度情形旨在优化一个标量目标函数。
- 问：可以在表格Q 学习中使用经验回放吗?
  - 答：可以，而且样本效率更高（为什么？）

深度Q 学习

伪代码：深度Q 学习（离策略版本）

初始化： 主网络和目标网络，初始参数相同。
目标： 从给定行为策略π_b 生成的经验样本中，学习近似最优动作值的最优目标网络。

将π_b 生成的经验样本存储在回放缓冲区$\mathcal{B} = \{(s, a, r, s')\}$ 中
    对每次迭代，执行：
        从$\mathcal{B}$ 中均匀抽取一个小批量样本
        对每个样本(s, a, r, s')，计算目标值$y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$，其中$w_T$ 为目标网络参数
        用小批量样本更新主网络，最小化$(y_T - \hat{q}(s, a, w))^2$
    每隔C 次迭代令$w_T = w$

注：

- 为什么没有策略更新?
- 网络的输入输出与DQN 论文中有所不同。

深度Q 学习

伪代码：深度Q 学习（离策略版本）

初始化： 主网络和目标网络，初始参数相同。
目标： 从给定行为策略π_b 生成的经验样本中，学习近似最优动作值的最优目标网络。

将π_b 生成的经验样本存储在回放缓冲区$\mathcal{B} = \{(s, a, r, s')\}$ 中
    对每次迭代，执行：
        从$\mathcal{B}$ 中均匀抽取一个小批量样本
        对每个样本(s, a, r, s')，计算目标值$y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$，其中$w_T$ 为目标网络参数
        用小批量样本更新主网络，最小化$(y_T - \hat{q}(s, a, w))^2$
    每隔C 次迭代令$w_T = w$

注：

- 为什么没有策略更新?
- 网络的输入输出与DQN 论文中有所不同。

深度Q 学习

伪代码：深度Q 学习（离策略版本）

初始化： 主网络和目标网络，初始参数相同。
目标： 从给定行为策略π_b 生成的经验样本中，学习近似最优动作值的最优目标网络。

将π_b 生成的经验样本存储在回放缓冲区$\mathcal{B} = \{(s, a, r, s')\}$ 中
    对每次迭代，执行：
        从$\mathcal{B}$ 中均匀抽取一个小批量样本
        对每个样本(s, a, r, s')，计算目标值$y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$，其中$w_T$ 为目标网络参数
        用小批量样本更新主网络，最小化$(y_T - \hat{q}(s, a, w))^2$
    每隔C 次迭代令$w_T = w$

注：

- 为什么没有策略更新?
- 网络的输入输出与DQN 论文中有所不同。

深度Q 学习

示例：

- 需要为每个状态-动作对学习最优动作值。
- 一旦获得最优动作值，即可立即得到最优贪心策略。

深度Q 学习

实验设置:

- 使用单个回合训练网络。
- 该回合由如图(a) 所示的探索性行为策略生成。
- 该回合仅有1000 步！ 而表格Q 学习需要100000 步。
- 使用含单隐层的浅层神经网络作为$\hat{q}(s, a, w)$ 的非线性近似器，隐层有100 个神经元。

详细内容见教材。

深度Q 学习

实验设置:

- 使用单个回合训练网络。
- 该回合由如图(a) 所示的探索性行为策略生成。
- 该回合仅有1000 步！ 而表格Q 学习需要100000 步。
- 使用含单隐层的浅层神经网络作为$\hat{q}(s, a, w)$ 的非线性近似器，隐层有100 个神经元。

详细内容见教材。

深度Q 学习

实验设置:

- 使用单个回合训练网络。
- 该回合由如图(a) 所示的探索性行为策略生成。
- 该回合仅有1000 步！而表格Q 学习需要100000 步。
- 使用含单隐层的浅层神经网络作为$\hat{q}(s, a, w)$ 的非线性近似器，隐层有100 个神经元。

详细内容见教材。

深度Q 学习

实验设置:

- 使用单个回合训练网络。
- 该回合由如图(a) 所示的探索性行为策略生成。
- 该回合仅有1000 步！而表格Q 学习需要100000 步。
- 使用含单隐层的浅层神经网络作为$\hat{q}(s, a, w)$ 的非线性近似器，隐层有100 个神经元。

详细内容见教材。

深度Q 学习

实验设置:

- 使用单个回合训练网络。
- 该回合由如图(a) 所示的探索性行为策略生成。
- 该回合仅有1000 步！而表格Q 学习需要100000 步。
- 使用含单隐层的浅层神经网络作为$\hat{q}(s, a, w)$ 的非线性近似器，隐层有100 个神经元。

详细内容见教材。

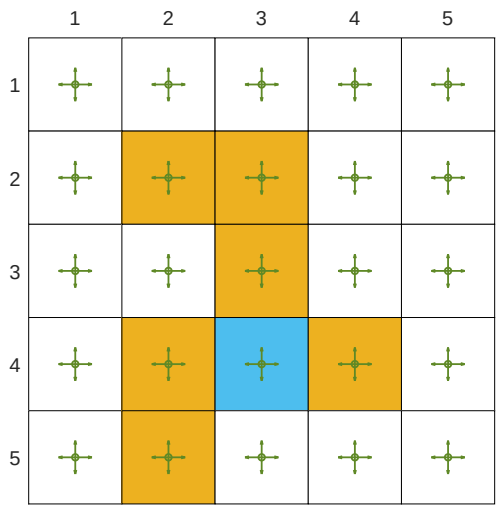
深度Q 学习

实验设置:

- 使用单个回合训练网络。
- 该回合由如图(a) 所示的探索性行为策略生成。
- 该回合仅有1000 步！而表格Q 学习需要100000 步。
- 使用含单隐层的浅层神经网络作为$\hat{q}(s, a, w)$ 的非线性近似器，隐层有100 个神经元。

详细内容见教材。

深度Q 学习



行为策略。

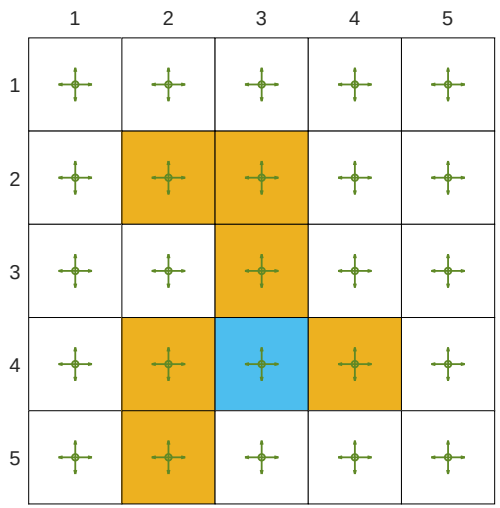
1000 步的回合。

得到的策略。

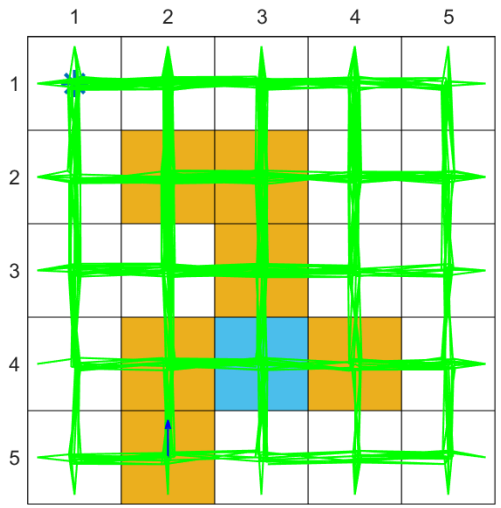
TD 误差收敛到零。

状态估计误差收敛到零。

深度Q 学习



行为策略。



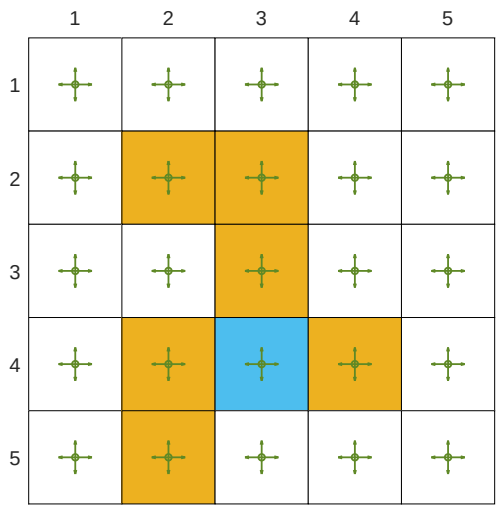
1000 步的回合。

得到的策略。

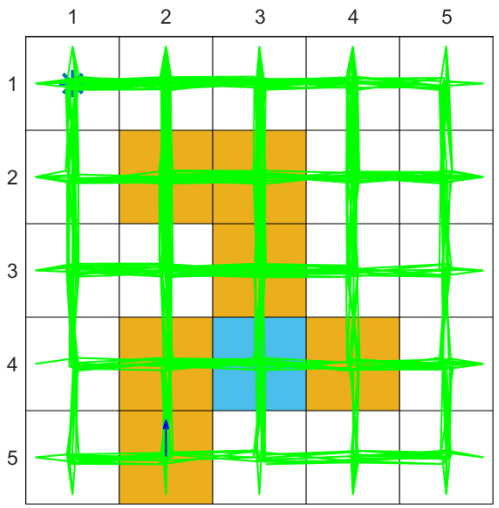
TD 误差收敛到零。

状态估计误差收敛到零。

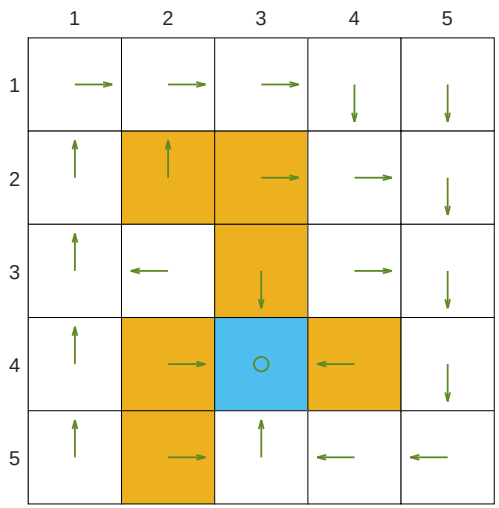
深度Q 学习



行为策略。



1000 步的回合。

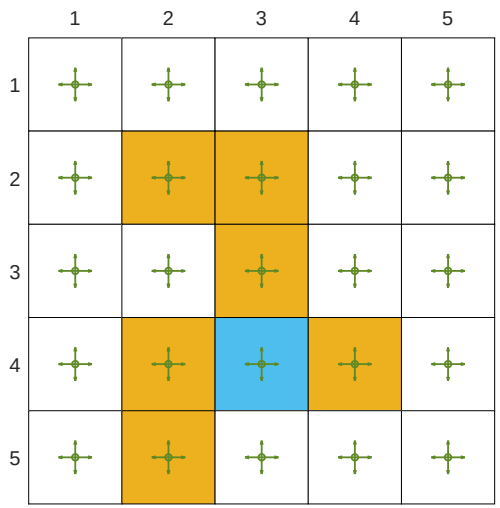


得到的策略。

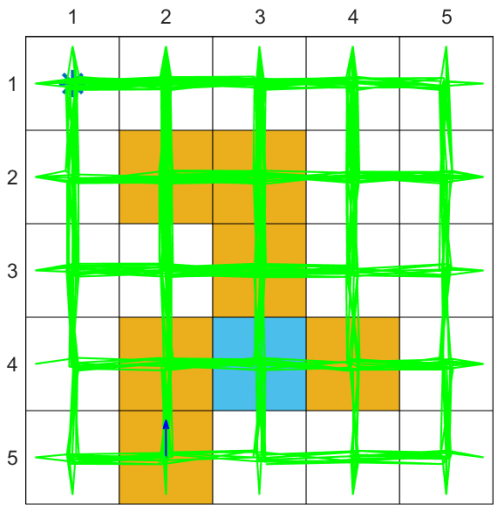
TD 误差收敛到零。

状态估计误差收敛到零。

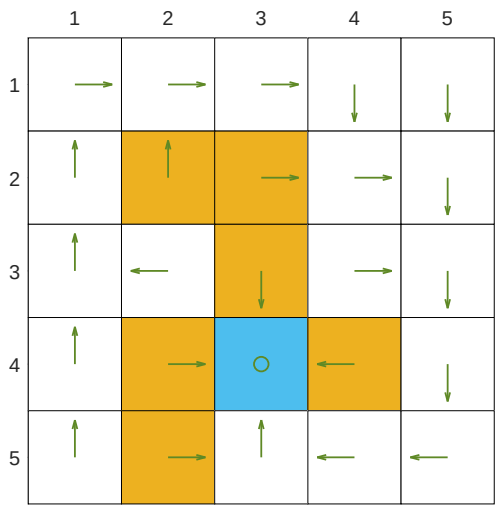
深度Q 学习



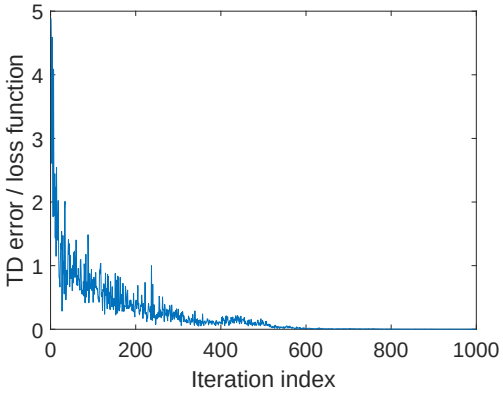
行为策略。



1000 步的回合。



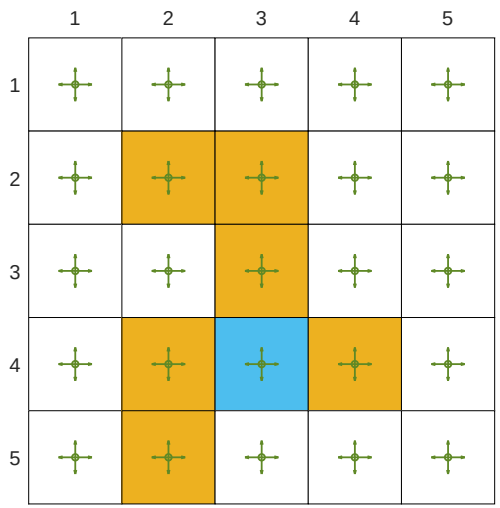
得到的策略。



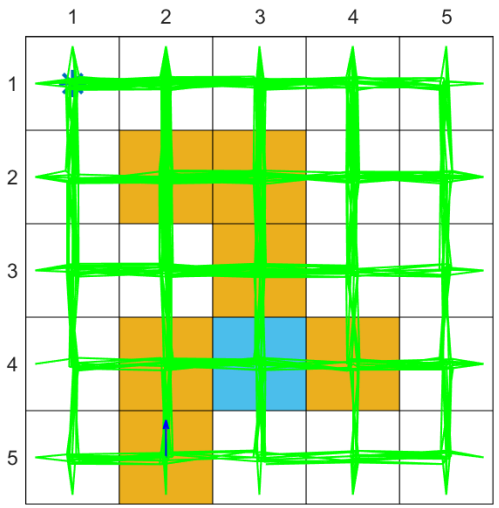
TD 误差收敛到零。

状态估计误差收敛到零。

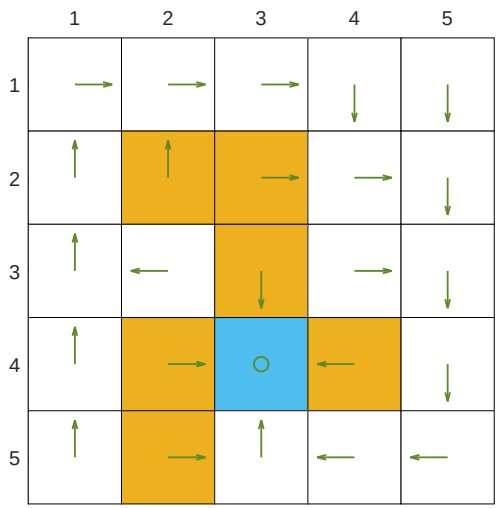
深度Q 学习



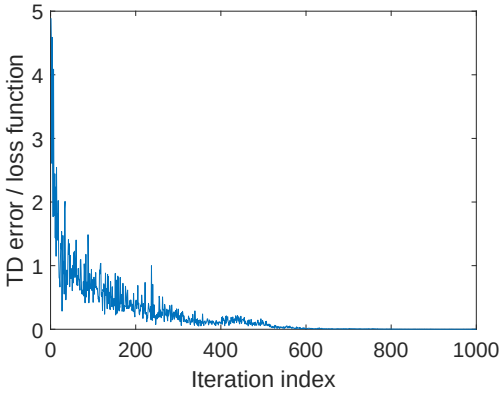
行为策略。



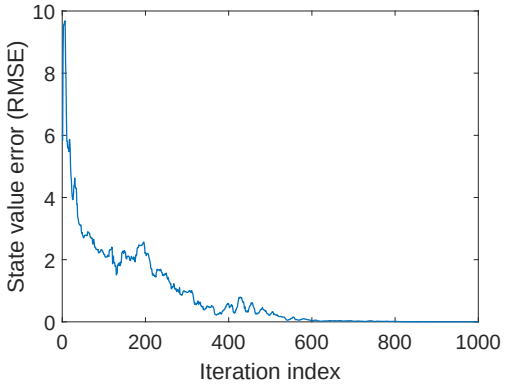
1000 步的回合。



得到的策略。

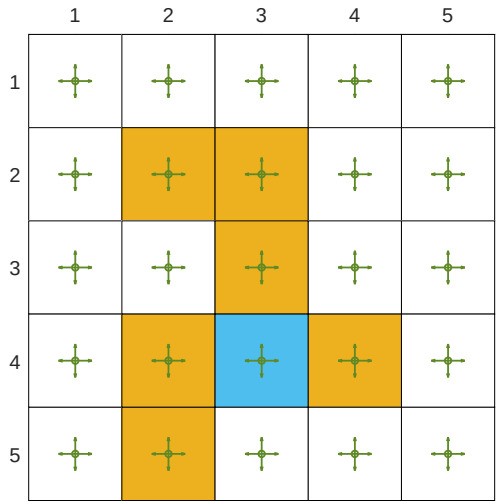


TD 误差收敛到零。



状态估计误差收敛到零。

若只使用100 步的单个回合会怎样？ 数据不足



行为策略。

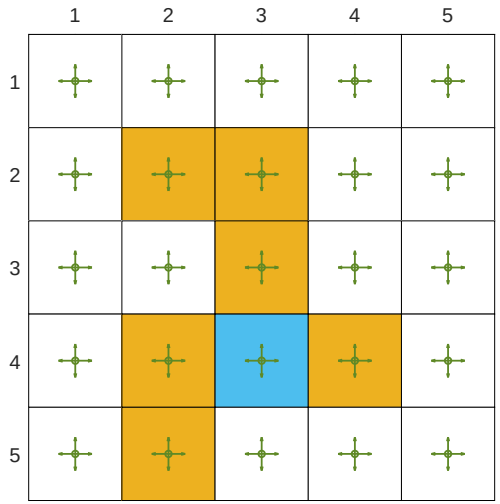
100 步的回合。

最终策略。

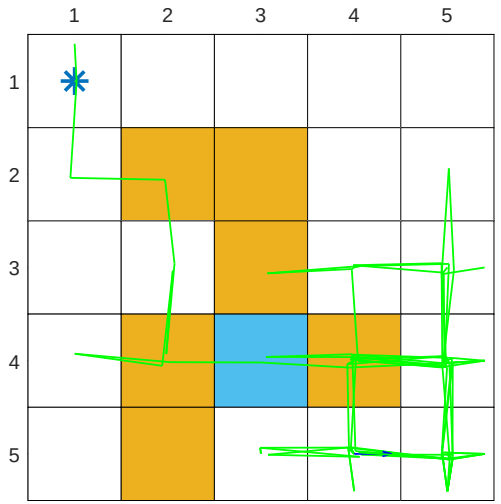
TD 误差收敛到零。

状态误差未收敛到零。

若只使用100 步的单个回合会怎样？ 数据不足



行为策略。



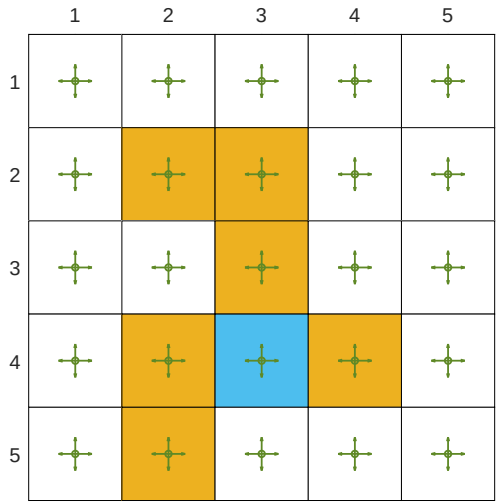
100 步的回合。

最终策略。

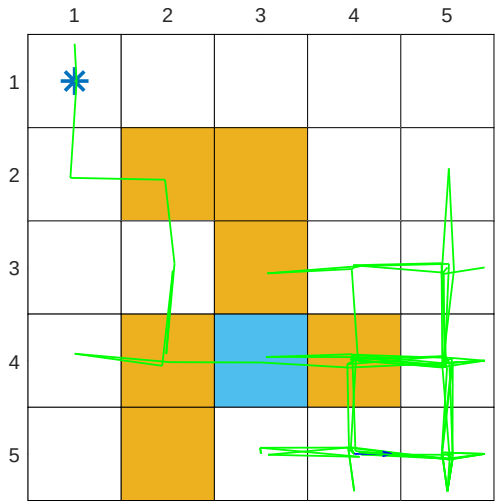
TD 误差收敛到零。

状态误差未收敛到零。

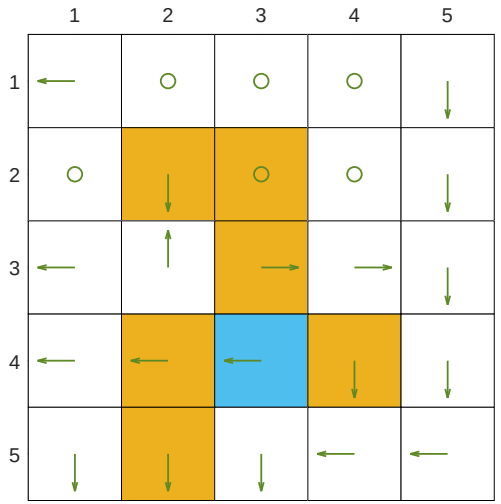
若只使用100 步的单个回合会怎样？ 数据不足



行为策略。



100 步的回合。

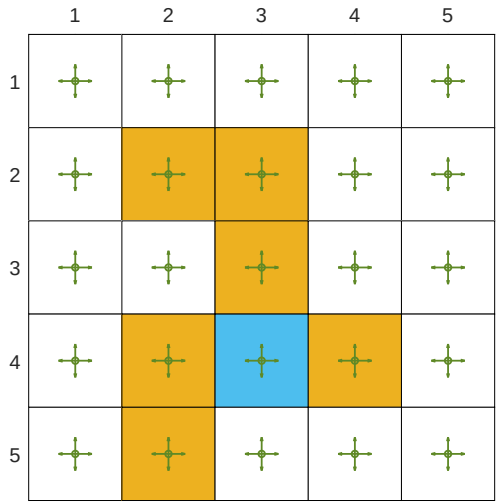


最终策略。

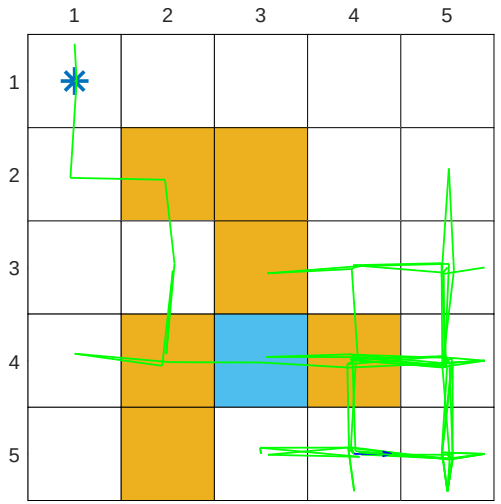
TD 误差收敛到零。

状态误差未收敛到零。

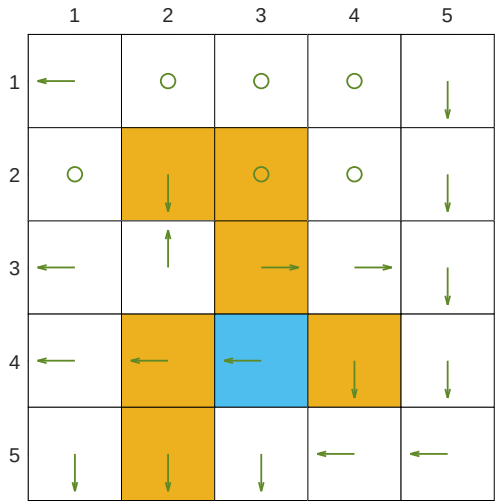
若只使用100 步的单个回合会怎样？ 数据不足



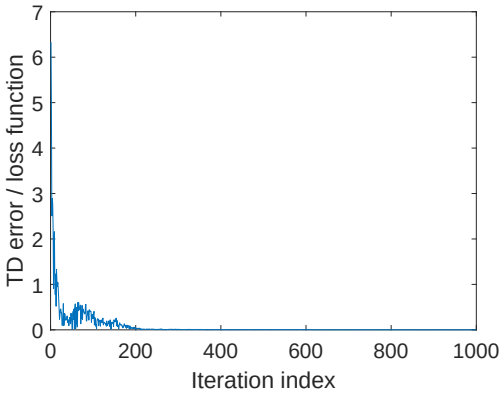
行为策略。



100 步的回合。



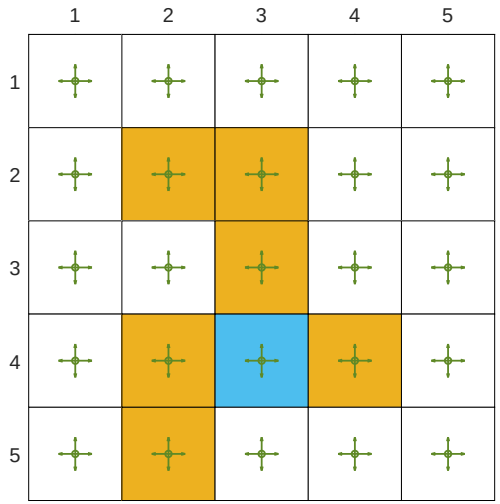
最终策略。



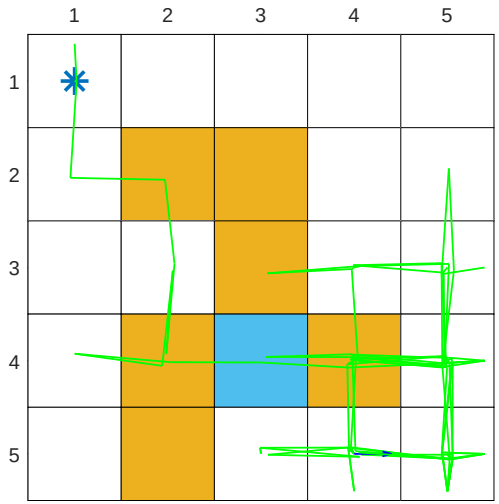
TD 误差收敛到零。

状态误差未收敛到零。

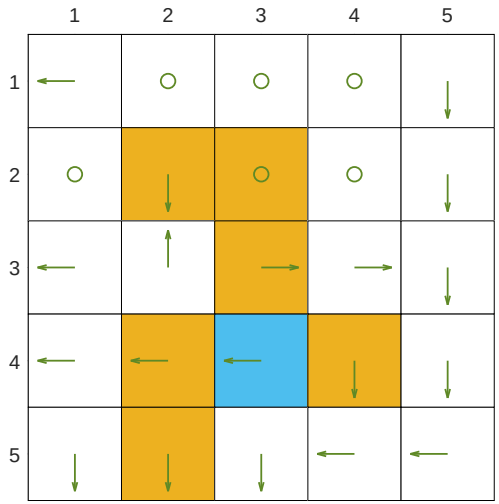
若只使用100 步的单个回合会怎样？ 数据不足



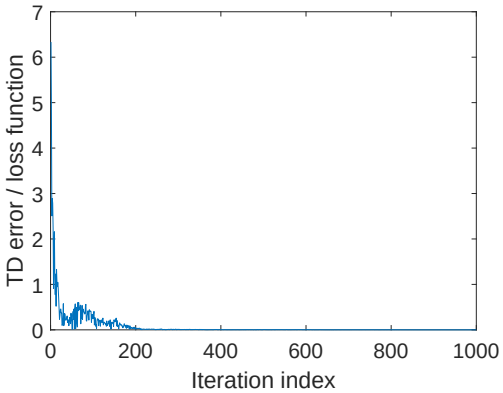
行为策略。



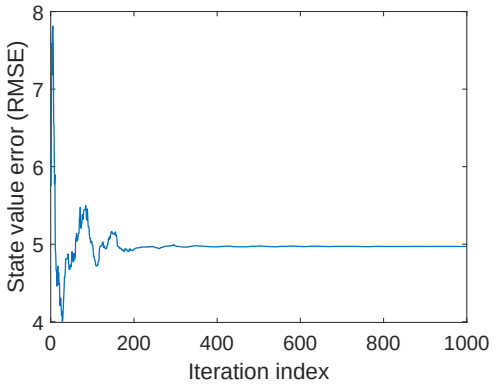
100 步的回合。



最终策略。



TD 误差收敛到零。



状态误差未收敛到零。

大纲

1 动机示例：从表格到函数

2 状态值估计算法
- 目标函数
- 优化算法
- 函数近似器的选择
- 示例
- 故事总结
- 理论分析（选读）

3 带函数近似的Sarsa

4 带函数近似的Q 学习

5 深度Q 学习

6 总结

总结

本讲介绍了值函数近似方法。

- 首先，理解基本思想。
- 其次，理解基本算法。